



UNIVERSITÀ DI PISA

MSC in Artificial Intelligence and Data Engineering

Large Scale and Multi-structured Databases

Discover Europe

Students:

Beatrice Castaldo
Francesca Santoro
Giusy Tucci

ANNO ACCADEMICO 2025–2026

Abstract

Embark on your next European adventure with a platform designed to inspire your wanderlust. Our application offers a vast portfolio of travel packages that users can easily navigate by trending popularity or through highly personalized “For You” recommendations, which are dynamically curated based on individual interests and age demographics. For those who want to dig deeper, the platform provides comprehensive destination breakdowns, featuring top-tier accommodations and exciting Points of Interest.

While this may sound familiar to standard booking sites, our application actively eliminates the notorious stress of trip planning. If aligning flight schedules, strict budgets, and seasonal logistics feels overwhelming, our interactive travel Questionnaire steps in to do the heavy lifting. By capturing your fundamental travel constraints, the system’s algorithm instantly cross-references your preferences with live flight data, generating complete, ready-to-book packages that perfectly fit your criteria.

For travelers eager to step out of their comfort zone, the platform introduces an innovative “Surprise Package” mode. This unique booking experience gamifies your vacation by intentionally masking your final destination and flight details. Armed with only a handful of mysterious clues, you will not discover exactly where you are heading until your departure date approaches, allowing you to fully embrace the thrill of the unknown.

Beyond the booking phase, your user dashboard acts as a centralized hub to seamlessly monitor your shopping cart and review your order history. Furthermore, once a trip is successfully confirmed, the application unlocks a specialized, graph-powered recommendation engine that suggests unvisited, highly-rated attractions situated just a few kilometers from your chosen hotel.

Ultimately, this platform redefines European travel planning, transforming it into an intelligent, effortless, and deeply exhilarating experience.

The code for the project is available on GitHub at the following link:

https://github.com/bcastaldo314/Discover_Europe

Contents

Abstract	i
1 Introduction	1
1.1 Key Concepts	1
2 Application Design	2
2.1 Actors	2
2.2 Requirements	3
2.2.1 Functional requirements	3
2.2.2 Non-functional Requirements	4
3 User Manual and Mockups	5
3.1 Login, Registration and User profile	5
3.2 Home Page	6
3.3 Questionnaire	7
3.4 Package Details	10
3.5 Cart and Flight Details	11
3.6 Travel History and Custom POIs	13
3.7 For You and Popular	15
3.8 Reviews	17
3.9 Administrator Statistics Dashboard	18
4 UML Class Diagram	20
5 Dataset, Data Retrieval and Data generation	21
5.1 Hotel Data Acquisition and Transformation	21
5.1.1 Geographical Filtering and Selection	21
5.1.2 Data Preprocessing and Schema Transformation	22
5.2 Flight Data Acquisition and Synthetic Generation	23
5.3 Points of Interest (POI) Extraction	23
5.4 Travel Packages Synthetic Generation	24
5.5 User Profiles Synthetic Generation	24
5.6 Questionnaires and Reviews Synthetic Data	24
5.6.1 Dataset Statistics	25
6 DB Design	26
6.1 Document DB - MongoDB	26
6.1.1 Collections	26
6.2 GraphDB - Neo4j	31
6.2.1 Graph Nodes	31
6.2.2 Graph Relationships	31

7	Implementation	33
7.1	Spring Framework	33
7.2	Config	33
7.3	Event	34
7.4	Model	34
7.4.1	MongoDB Models	35
7.4.2	Neo4j Graph Models	44
7.5	Repository	47
7.5.1	MongoDB Repositories	47
7.5.2	Neo4j Graph Repositories	48
7.6	Service	49
7.6.1	MongoDB Services	49
7.6.2	Neo4j Graph Services & Distributed Synchronization	50
7.7	Controller	51
7.8	Utils	52
8	Relevant Queries	54
8.1	MongoDB Queries	54
8.1.1	Retrieve Negative Reviews per Package	54
8.1.2	Surprise Package Success Rate	56
8.1.3	Average Budget by Age Range	57
8.1.4	Monthly Revenue and Order Volume Analysis	58
8.1.5	Top Customers Loyalty Campaign	59
8.2	Neo4j Queries	60
8.2.1	Popular Packages Feed Generation	60
8.2.2	Demographic Filtering Recommendations	61
8.2.3	Interest-Based Filtering Recommendations	63
8.2.4	Top Destinations by Age Range	65
8.2.5	Geospatial Point of Interest (POI) Discovery	67
9	Indexes	69
9.1	MongoDB Indexes	69
9.1.1	Reviews Collection	69
9.1.2	Flights Collection	70
9.1.3	Travel Packages Collection	71
9.2	Neo4j Graph Database Indexes	71
9.2.1	User_Id_Idx	71
9.2.2	User_Birth_Idx	72
9.3	Sharding	73
9.3.1	MongoDB sharding	73
9.4	Deployment and Replica Set Configuration (Local Environment)	74
9.4.1	MongoDB	74
9.4.2	Neo4j	75
9.4.3	CAP Theorem Considerations	75
9.4.4	Handling Inter-DB Consistency	76
10	Use of Artificial Intelligence in Development	78
10.1	Overview	78
10.1.1	Support in Database Population	78
10.1.2	Java and Spring Boot Architecture	78
10.1.3	Database Synchronization Strategy	79
10.1.4	MongoDB and Neo4j Query Optimization	79

Chapter 1

Introduction

Discover Europe is a modern application designed for spontaneous adventurers and travelers eager to experience the beauty of Europe without the traditional stress of trip planning. The primary goal of the platform is to simplify, accelerate, and enhance the travel booking process by providing a highly personalized, data-driven itinerary catalog coupled with real-time flight integrations.

The platform is built around two main types of users: **Customers**, who seek to discover new destinations, optimize their travel budget, and embrace thrilling surprise adventures; and the **Admin**, who manages the ecosystem to ensure high-quality standards, track platform revenue, and monitor demographic travel trends.

Ultimately, the system is designed to streamline the entire vacation preparation experience, making travel more engaging and accessible for all users, while relying on a robust persistence architecture (MongoDB and Neo4j) to deliver fast recommendations.

1.1 Key Concepts

The main key concepts of the application are:

- **Dynamic Profiling and Suggestions:** Customers can submit a detailed travel questionnaire (specifying budget limits, preferred seasons, travel styles and so on). The system's algorithmic engine instantly cross-references these preferences with live flight availability to generate perfectly matching, ready-to-book travel packages.
- **Surprise Travel Experience:** A unique feature that allows users to book mystery trips. The application dynamically censors the destination city, hotel and flight details across the entire user interface (including the cart and order history), revealing them only when the travel date approaches.
- **Graph-Powered Recommendations:** Leveraging advanced graph database traversals, the platform provides highly personalized "For You" sections. It utilizes age-based filtering and interest-based filtering to suggest the most relevant travel experiences.
- **Advanced Application Analytics:** The platform provides deep business insights, enabling the Admin to track rolling monthly revenue, automatically distribute promotional discount codes to top-spending customers, monitor destination popularity by age group, and proactively detect negative feedback via keyword-based semantic analysis.

Chapter 2

Application Design

2.1 Actors

The main actors of DiscoverEurope are:

- **Unregistered User:** The system must allow unregistered users to become customers by registering with their first name, last name, phone number, nationality, birth date, email, and password. They can also insert the details of their documents (such as ID card and passport) while registering in the app, or they can decide to do it later, when already customers.

- **Customer:** The customer can visualize and update the profile. They can answer a few questions in the questionnaire (user interests, favourite season, budget, city of departure, user's age range, surprise or not) and choose among three packages that respect the user's choices. The customer can see the details of the packages if they didn't choose the surprise option, see their reviews and eventually add one to the cart.

If the client chose the surprise option in the questionnaire, the system will output three censored packages and the customer will only be able to add one to the cart. The surprise package must be censored in the travel history as well, and the system must give the user clues to guess the destination. The system must reveal the package details, along the destination, 3 days before the departure date, allowing the user to prepare for their trip.

The customer can either delete a package from the cart or proceed with the check out. Once a package has been checked out, the system must allow the user to visualize the package in the Travel History, where the customer can add activities and reviews to the package, and see the flight details. The system must allow the user to view travel packages recommended based on past experiences of other users similar to them, according to either age or interests.

The system must allow the user to view the most popular travel packages selected by other app users ("Popular" section).

The system must allow the user to submit reviews and feedback for their completed travel experiences.

- **Admin:** The system must allow the system administrator to manage all user accounts and to analyze statistics on bookings, most popular destinations, see which packages get the most negative feedback to improve them, see the total orders and revenue per month, see what kind of budget gets chosen by which age range of users, and so on.

2.2 Requirements

In the following section are listed the requirements of the application.

2.2.1 Functional requirements

Unregistered user

- The system must allow an unregistered user to become a registered customer by signing up with their first name, last name, phone number, email address, password, nationality, age, and identification document.
- The system must allow an unregistered user to log in as a registered user.

Customer

- The system must allow the user to log in.
- The system must allow the user to modify their account details (password, email, etc.).
- The system must allow the user to delete their account.
- The system must allow the user to retake the questionnaire for new trips.
- The system must allow the user to view their trip history.
- The system must allow the user to submit reviews/feedback on completed travel experiences.
- The system must allow the user to add travel packages to a cart.
- The system must allow the customer to view the items added to the cart.
- The system must allow the user to modify the cart and delete items if necessary.
- The system must allow the user to view travel packages recommended based on the past experiences of similar users, according to age or shared interests ("For You" section).
- The system must allow the user to view the most popular travel packages chosen by other app users ("Popular" section).
- The system must allow the user to answer a short questionnaire, including the following criteria: user interests, favourite season, budget, travel style, city of departure, user's age range, surprise or not.
- The system must provide the user with three surprise travel packages, without revealing specific details, generated based on the user's questionnaire responses.
- The system must send the user clues or "spoilers" prior to departure.
- The system must provide the final destination to the user only three days before departure, at which point the uncensored package details become viewable in the trip history.
- The system must allow the user to consult the provided points of interest once the destination is revealed.
- The system must provide the user with three travel packages based on the details provided in the questionnaire.

- In the event that these three packages do not meet the user's expectations, the user is redirected to the "Popular" section.
- If no trips matching the user's specific requirements are found, the system shall propose three similar packages that satisfy the user's core specifications (e.g., departure city, preferred season, travel style), utilizing a match score mechanism that prioritizes specific responses over others. In all cases, regardless of the match score, three packages will be provided in response to the user's questionnaire.
- The system must allow the user to initiate the checkout process, including the input of their address and preferred payment method.
- The system must allow the user to cancel an ongoing checkout process.
- The system must allow the user to view the final result/confirmation of the checkout process.

Admin

- The system must allow the system administrator to log in: the system administrator has a specific email address that automatically identifies them within the system.
- The system must allow the system administrator to manage all user accounts.
- The system must allow the administrator to analyze customer questionnaires and profiles.
- The system must allow the system administrator to analyze statistics on bookings, most popular destinations, see which packages get the most negative feedback to improve them, see the total orders and revenue per month, see what kind of budget gets chosen by which age range of users, and so on.
- The system must allow the system administrator to view a customer's travel history and reviews in order to better personalize future offers.

2.2.2 Non-functional Requirements

- The system must be developed using an Object-Oriented Programming (OOP) language (Java).
- The system must implement a multi-database architecture utilizing a DocumentDB and a GraphDB.
- The system must securely encrypt the users' passwords.
- The system must be a well-responsive website application.
- The system must communicate with any user via a secure communication channel (HTTP).
- The code of the application must be easily readable and maintainable.
- The application must be intuitive and user-friendly.
- The system must expose all its functionalities by means of well-documented RESTful APIs.

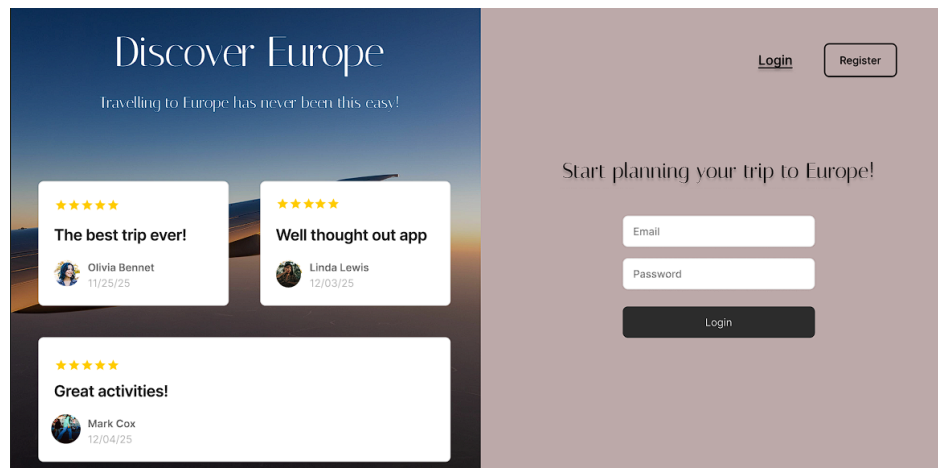
Chapter 3

User Manual and Mockups

3.1 Login, Registration and User profile

The **Login page** allows any user to authenticate themselves, through their e-mail and password. The system automatically checks their identity to verify the entered credentials, ensuring secure access before redirecting the user to their customized personal area.

If an unregistered user wants to access the application, they will have to click on the button on the top right corner to access the registration page.



The **Registration page** allows an unregistered user to create an account entering their information. The unregistered user is required to enter first name, last name, phone number, nationality, birth date, e-mail address and password. The unregistered user can optionally upload the identification documents' information during the registration phase, otherwise they will be able to modify their profile later on.

Discover Europe Register Login

Join us

First name Fortunata	Last name Mancini	Phone +39 0941358203	Nationality Italy
Birthdate 04/01/1957	Email address fortunata.mancini458@fastwebnet.it	Password *****	

Register

To upload your identity documents now, click [here](#)

The User profile page can be accessed by registered and unregistered users, to either modify their password or update their document information.

Discover Europe Your Profile Home

Your Profile

First name Fortunata	Last name Mancini	Phone +39 0941358203	Nationality Italy
Birthdate 04/01/1957	Email address fortunata.mancini458@fastwebnet.it	Password *****	
Passport Number LY0190579	Passport Expiration Date 10/11/2035	ID Number LY0190579	ID Expiration Date 10/11/2035

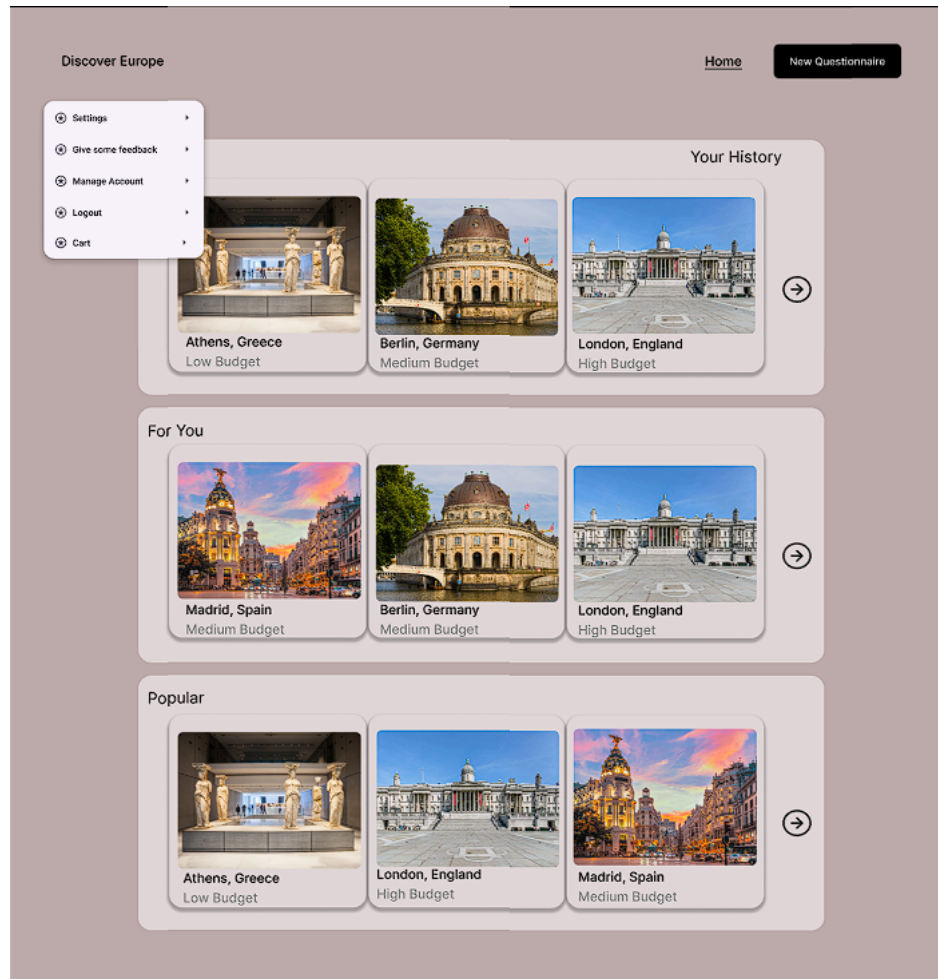
Update Profile Delete Account

3.2 Home Page

The Home page can be accessed by customers, where they can see the entry points to their Travel History, For You page and Popular page.

Moreover, on the top right corner, clicking on the New Questionnaire button, the user can compile a new questionnaire and eventually choose a new Travel Package.

Through the drop down menu in the Home page, the user can also access to their profile ("*Manage account*"), access their cart and write a new review on a previously purchased package ("*Give some feedback*").



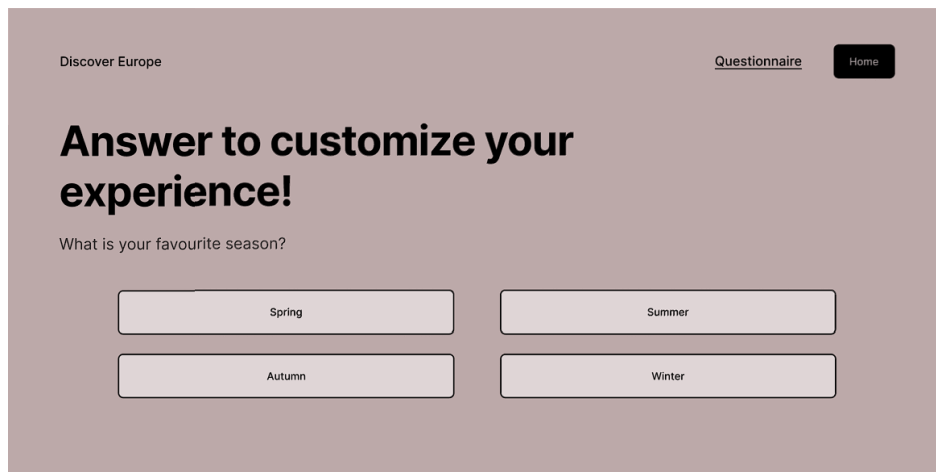
3.3 Questionnaire

As previously stated, clicking the New Questionnaire button from the Home Page initiates the questionnaire flow. The questionnaire is structured as 7 sequential questions specifically designed to construct the user's travel profile. The wizard collects the following data points:

- **User Interests** (Multi-select): captures the user's core focus for the trip, allowing the system to retrieve packages with matching attributes (e.g., selecting *Culture* prioritizes itineraries featuring museums and historical landmarks). The user can choose between Culture, Nature, Nightlife, Relax, Food or Shopping. Each matched interest adds 10 points to the total match score, up to a maximum of 30 points.
- **Preferred Season** (Single-select): defines the desired travel timeframe. A seasonal match between the user's input and the package attributes carries the highest weight in the scoring algorithm, contributing 40 out of 100 points. Furthermore, this parameter is essential for querying valid flight schedules.
- **Budget Limit** (Single-select): acts as a strict hard filter; any travel package whose base price exceeds the user's specified ceiling is immediately excluded from the candidate pool.
- **Travel Style** (Single-select): allows the user to opt for *Low-cost*, *Standard*, or *Luxury*. A direct match with the package's tier adds the final 30 points to the match score.
- **User Age Bracket** (Single-select): collected primarily to feed recommendation algorithms, such as populating the personalized "*For You*" section.

- **Surprise** (Boolean): if set to **true**, the generated packages are masked: the destination city is hidden from the UI, and the user is provided solely with clues.
- **Departure Location** (String input): serves as a geographical hard filter. The system uses this location to eliminate incompatible flights and discard any package lacking a viable flight route from the chosen origin.

A couple of examples of these questionnaire pages will follow.



Discover Europe

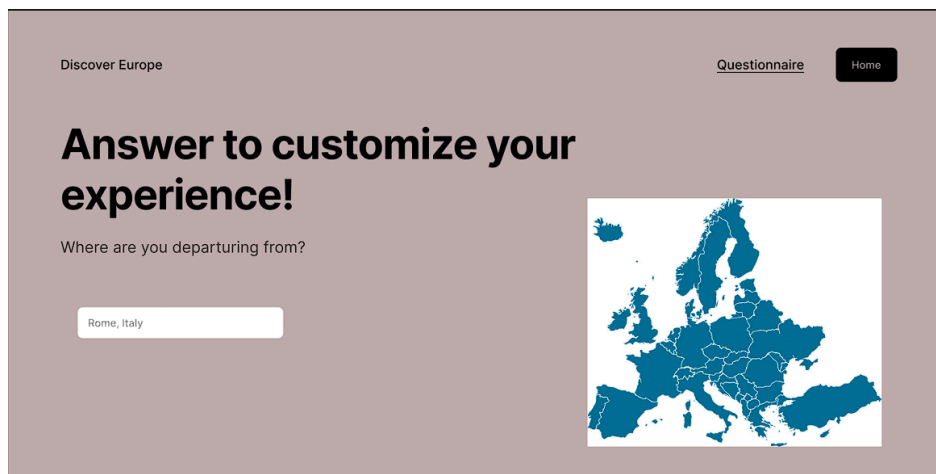
Questionnaire Home

Answer to customize your experience!

What is your favourite season?

Spring Summer

Autumn Winter



Discover Europe

Questionnaire Home

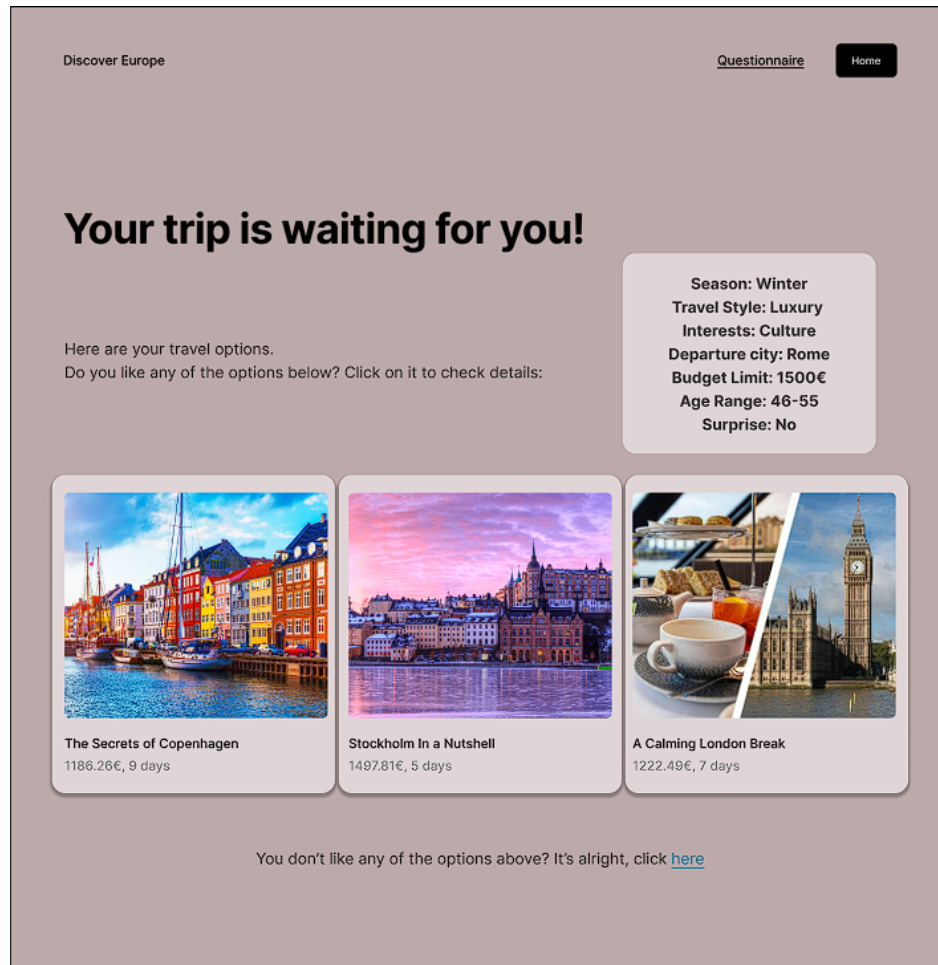
Answer to customize your experience!

Where are you departing from?

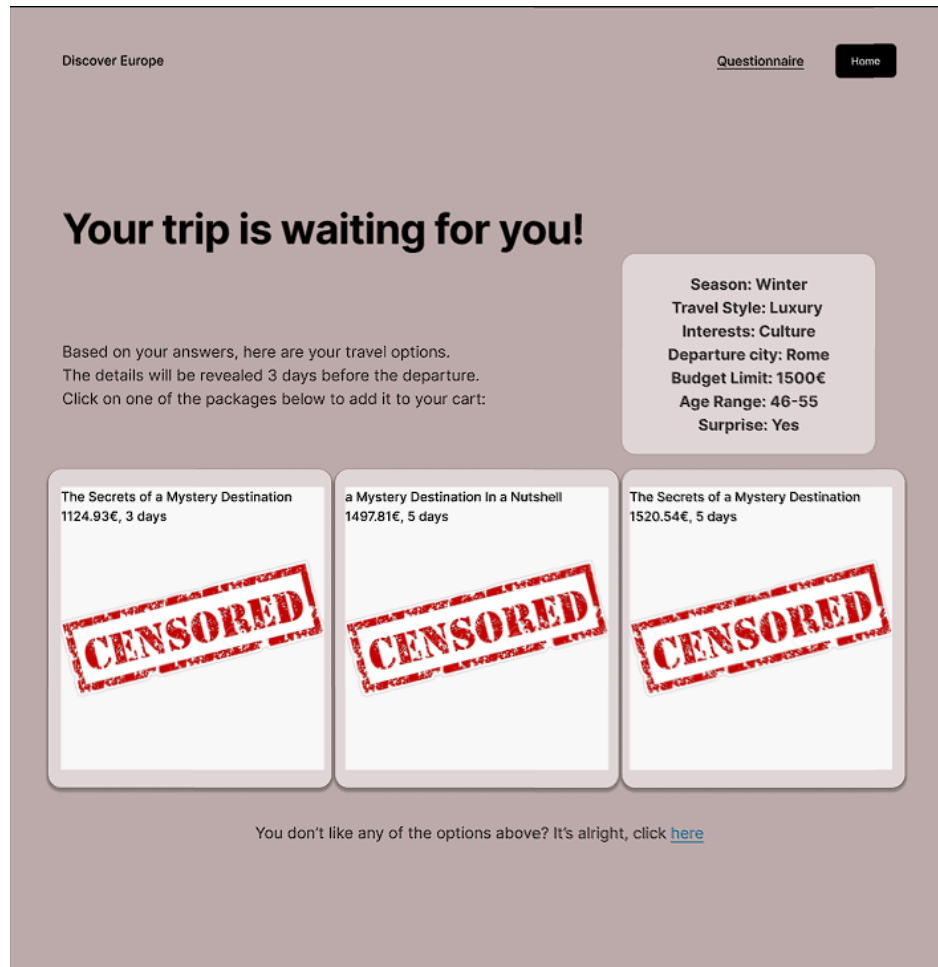
Rome, Italy



The output of the questionnaire when the user chose no surprise will be represented in the following image: three travel packages that have the highest match score the algorithm was able to find. The questionnaire selections are displayed on the top right. By clicking on one of the packages, the user will be able to see the details of the travel package.



Otherwise, if the user chose the surprise option, the output will be represented in the following image: three masked travel packages, which obviously were retrieved in the same way, with the highest match score. By clicking on one of them, the customer will be able to add it to their cart.



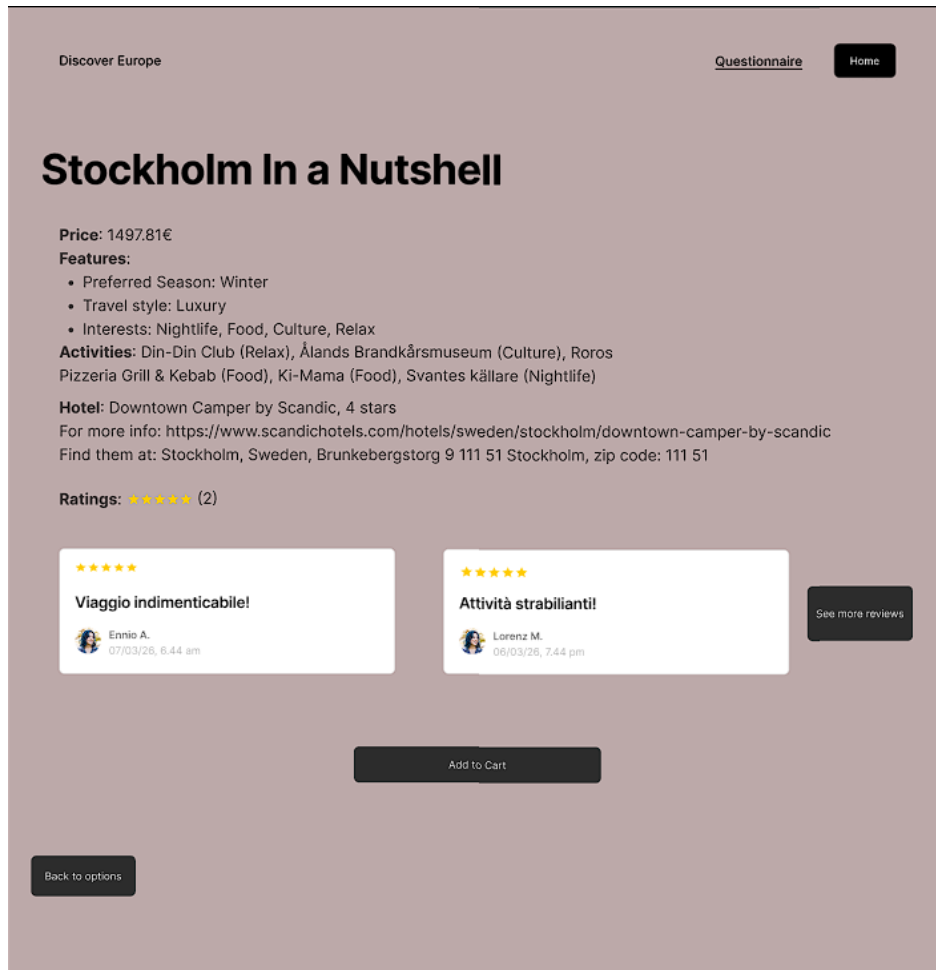
3.4 Package Details

Once the user chooses a package, either by completing a questionnaire or by selecting it from the For You/Popular pages, they can visualize every detail of said package in the Package Details page. Specifically, the information that the browsing customer is able to see in this page is:

- **price**;
- **package features**, such as preferred season, travel style, and interests, which, as discussed above, contribute to the match score;
- **activities**, which category indicated in brackets must match with the interests chosen by the user;
- **hotel information**, specifically the name, stars rating, address, and website;
- the average **rating** of the package, specifying the total number of reviews, and shown below are the last three reviews sorted by time.

Finally, the customer can click on the "See more reviews" button to check every review ever written on the package, and through the "Add to Cart" button they can add that specific package to the list of items in the cart and will be redirected to the cart page. If the details aren't convincing enough, the user can always go back to check if the other two options are better. Otherwise, if none of the package options are satisfying for the user taste, clicking on

the blue link below the three packages will redirect the customer to the Home page, where they will find the entry point to the For You page and Popular page.

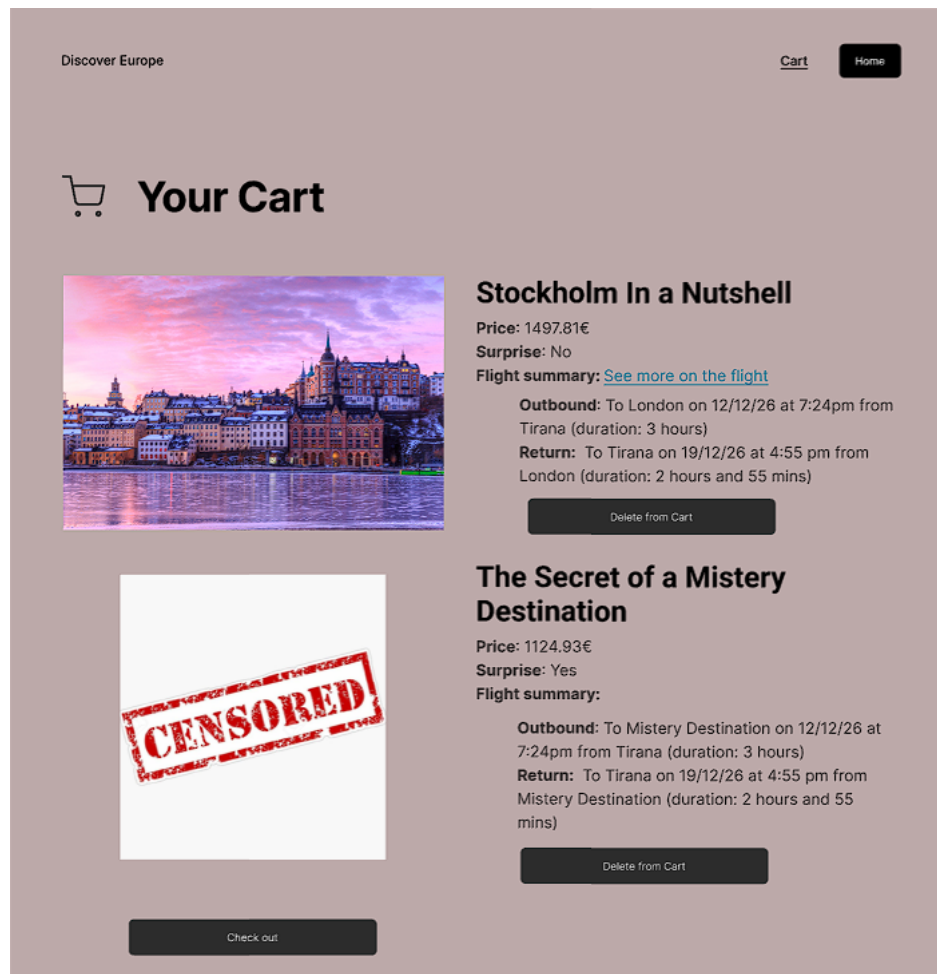


3.5 Cart and Flight Details

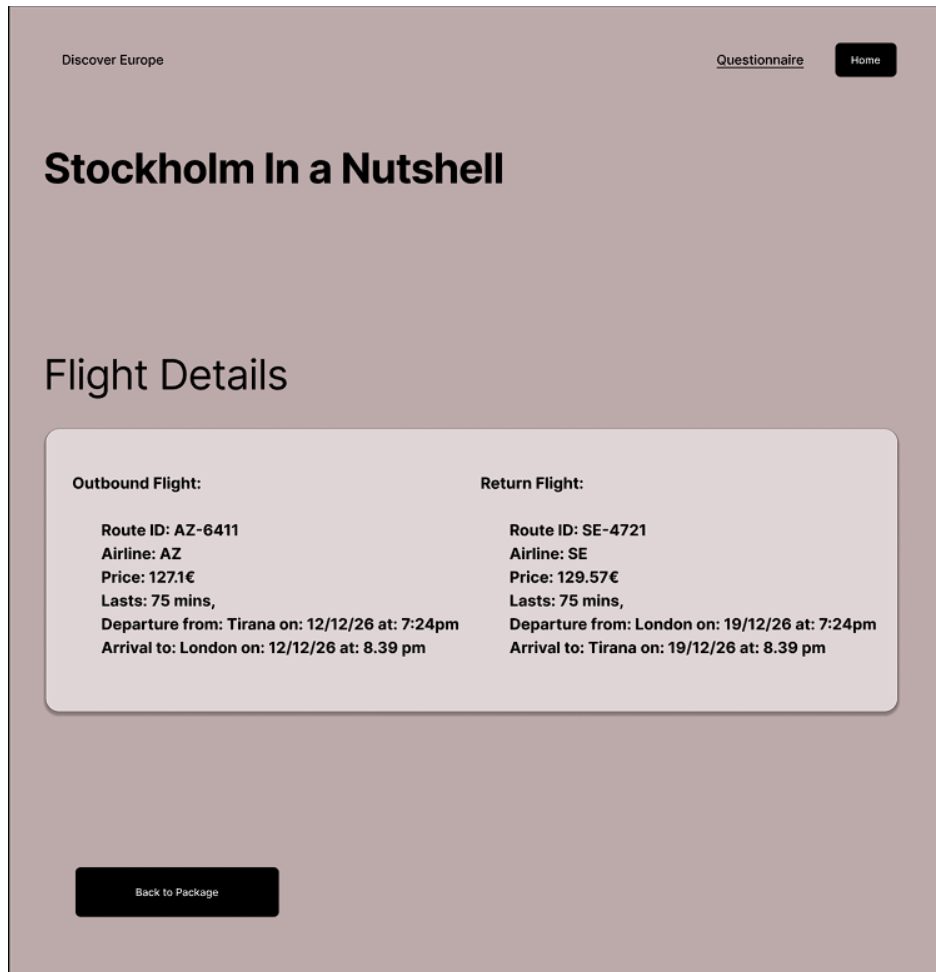
The cart contains every package inserted by the user. In particular, for the no surprise travel packages, from this page the customer can access the flight details, while already seeing a brief summary of it, also being able to visualize the title of the package and the price.

In a situation where the user chose the surprise option, most of the information in the cart will be masked, including the image, the title, and the destination of the flight summary. In this scenario, the user will not be able to explore the flight details, since it would spoil the surprise, but still being able to see the fundamental information to organize themselves, like date and time of departure and return.

At any point, the user will be able to delete any package from their cart through the apposite button.



To visualize the flight details, there is a dedicated page with every possible information on the flight, in particular route ID, airline, price, duration and location, time and date of departure and arrival, both for the outbound and the return flight.

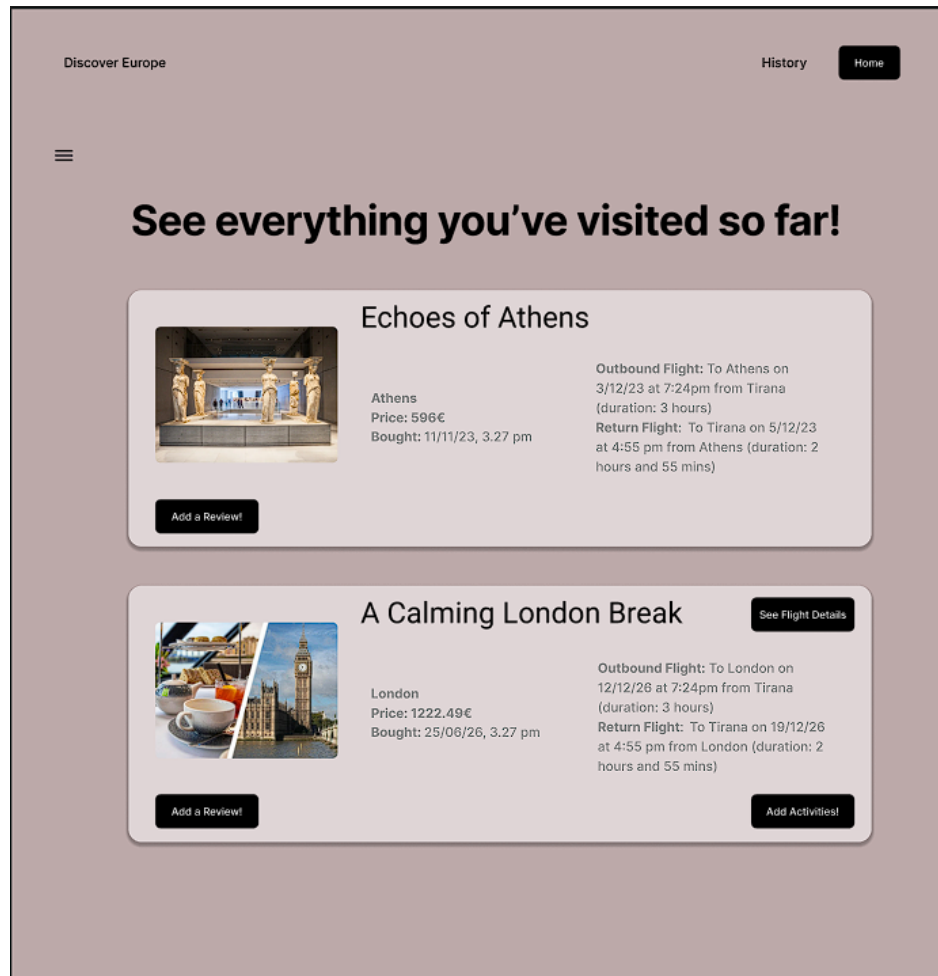


3.6 Travel History and Custom POIs

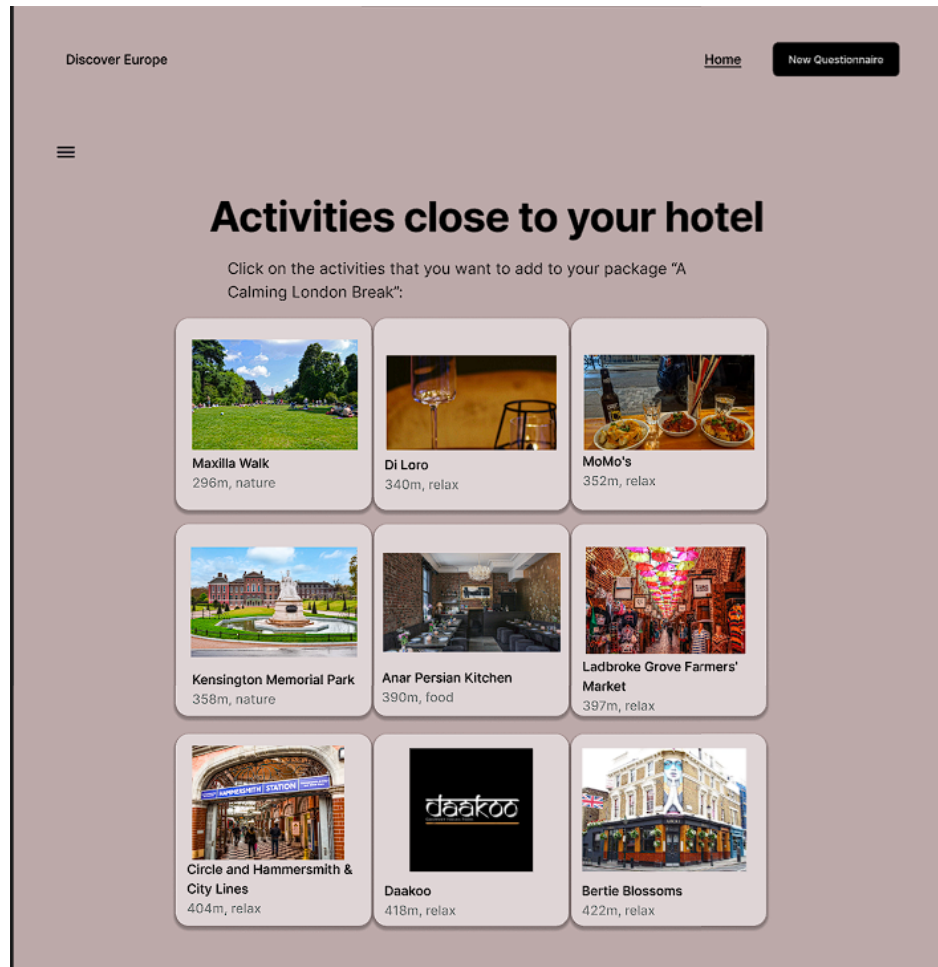
From the Home Page, the user can get to the Travel History, which contains every order the customer ever made. The single order includes the destination, price, date and time of purchase, and a brief flight summary.

If the travel is already finished, there will be a button to leave a review, otherwise there will be also two other buttons, leading to the Flight Details page and the Custom POIs page.

Moreover, if the travel package was just bought and the surprise option was selected, the details about the destination will be masked, just as it happened in the cart.



The Custom POIs page presents the user with the possibility of adding activities to their already purchased travel package, just for exploring the available possibilities. In fact, this page shows the customer the most interesting Points of Interests in the city of the destination, sorted by ascending distance with respect to the hotel of the travel package.

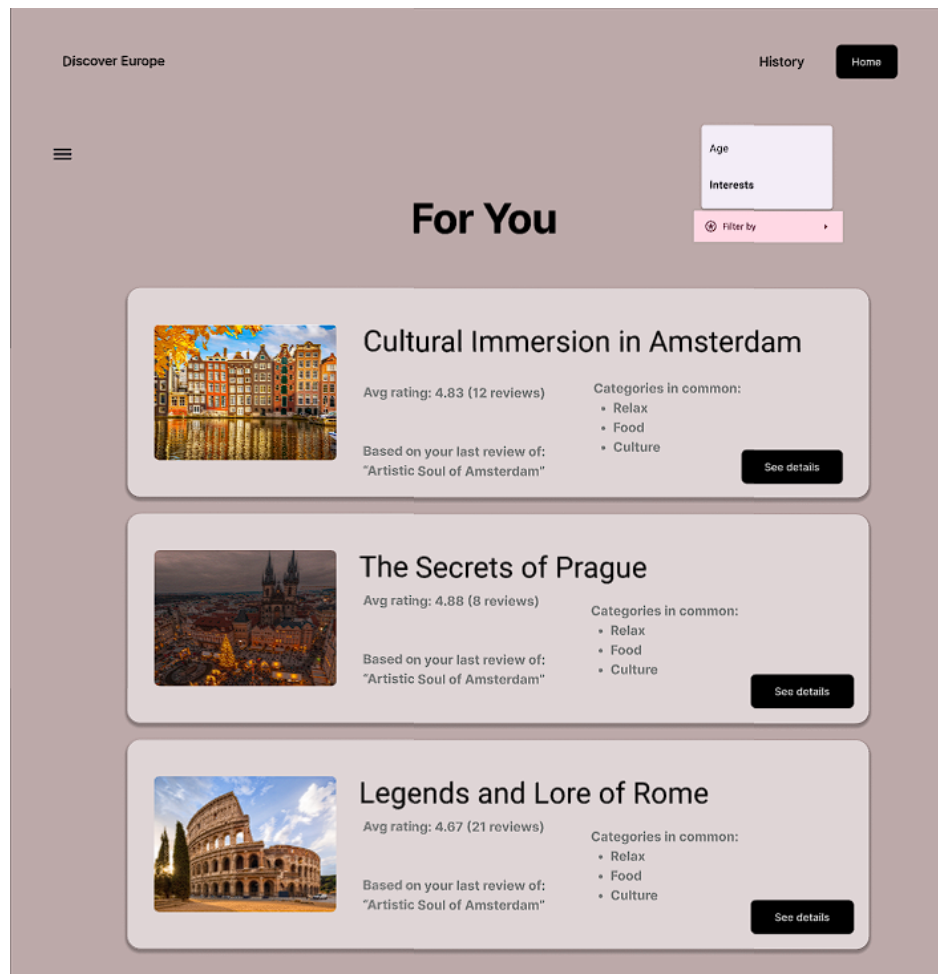


3.7 For You and Popular

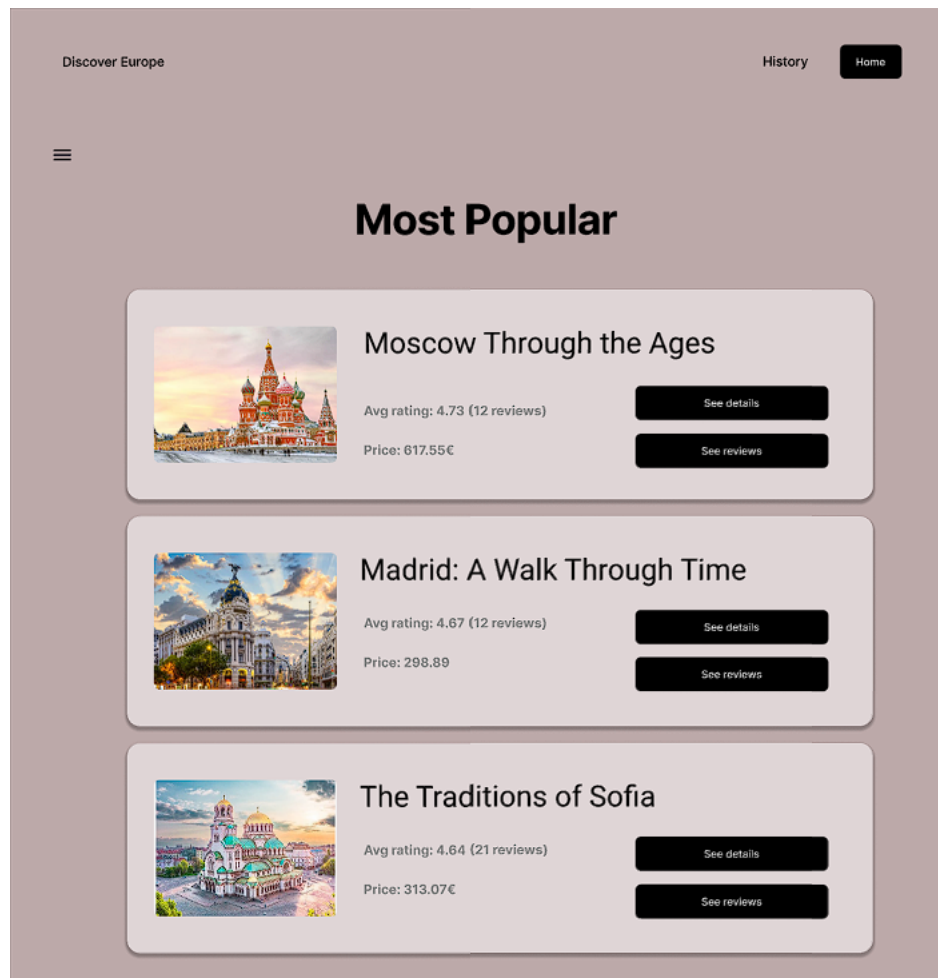
The For You page presents the user with interesting possibilities of new travel packages, based on the similarities to other users. This similarity is measured either by age range or interests in common.

Filtering for one or the other, the user can already see some details of these never seen before travel packages, such as average rating, the reason why it's recommended and the characteristics in common between different users.

Clicking on the button on the bottom right, the customer can get to the Package Details page.



The Popular page, very similarly to the For You page, shows the packages that were the most popular between users in general, according to rating and purchases. Through this page, the user can access both the Package Details page and the Reviews page.



3.8 Reviews

In the Reviews page, accessible from many pages in the application, the user is able to visualize all the reviews made by other customers, and even leave a review themselves, by writing in the white bar below and clicking submit.

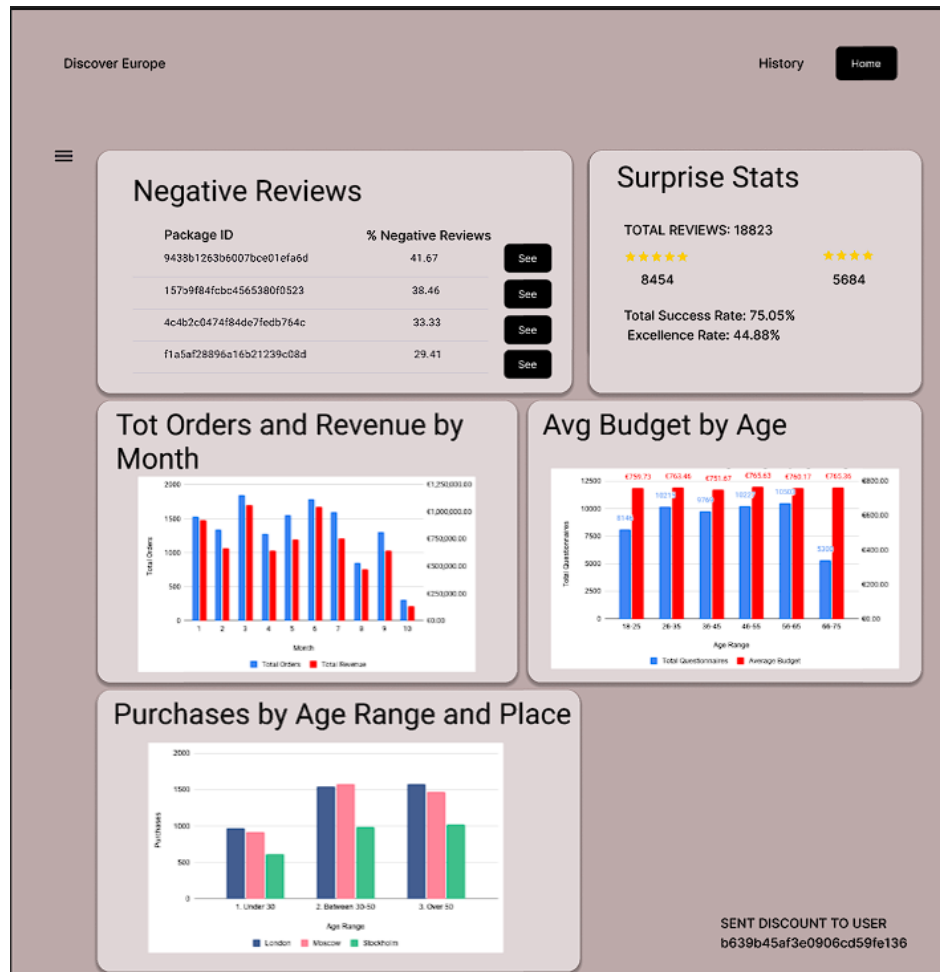
In the top part of the page, the customer can also see how many people left reviews on the specific travel package and its average rating.

The mockup shows a user interface for a travel platform. At the top, there's a header with 'Discover Europe' on the left, 'Questionnaire' in the center, and a 'Home' button on the right. The main title is 'Stockholm In a Nutshell'. Below it, the ratings are shown as five yellow stars and '(2)'. There are two review cards. The first card has a five-star rating, the title 'Viaggio indimenticabile!', a user profile picture, the name 'Ennio A.', and the date/time '07/03/26, 6:44 am'. The second card has a five-star rating, the title 'Attività strabilianti!', a user profile picture, the name 'Lorenz M.', and the date/time '06/03/26, 7:44 pm'. At the bottom, there's a section titled 'Leave your review:' with a text input field and a 'Submit' button.

3.9 Administrator Statistics Dashboard

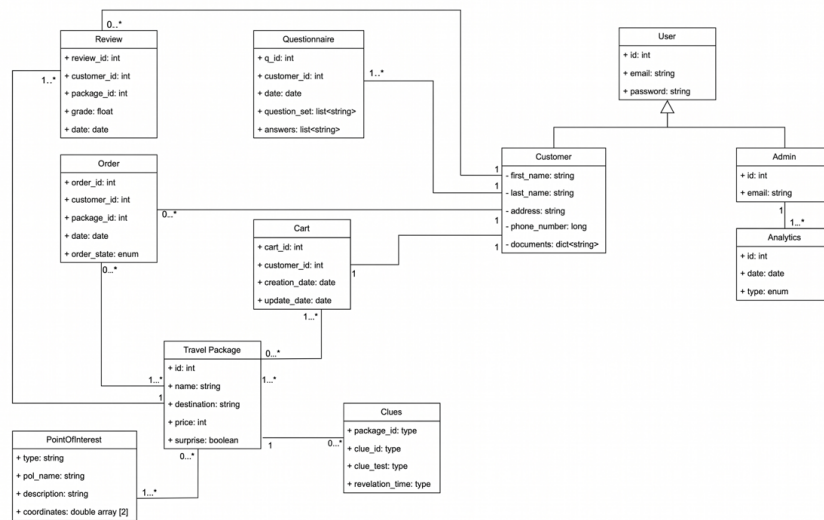
The Administrator Statistics Dashboard aggregates platform data mined from user questionnaires, completed bookings, and feedbacks into five analytical modules and a system log:

- **Negative Reviews:** here the admin can see which packages are performing worse, in descending order of dissatisfaction rates. Through the button "See" the admin can access that specific package and analyze what characteristics the customers are disliking.
- **Surprise Stats:** shows the success rate of surprise travel packages. As shown in the image, the packages have a pretty high success rate of 75%, with a respectable excellence rate of 44.8%.
- **Total orders and revenue by month**
- **Average budget by age:** shows the groups of people that choose a certain budget over others, to design travel packages better suited for everyone.
- **Purchases by age range and place:** shows the destinations that are preferred, and by which group of people, divided by age range.
- **10% Discounts to top customers**



Chapter 4

UML Class Diagram



Chapter 5

Dataset, Data Retrieval and Data generation

The development of the application Discover Europe required the acquisition and generation of different types of data related to users, questionnaires, hotels, flights, reviews, travel packages and points of interests relative to the multiple Europe capitals included in the application.

5.1 Hotel Data Acquisition and Transformation

As far as accommodation facilities are concerned, raw global hotel or B&B datasets available online were not adequate for our application in their native form, since they presented with either way too many fields that did not matter for the purposes of Discover Europe or less data than our application would require.

To ensure high data quality, we opted to construct a refined dataset starting from a publicly available repository, which was subsequently filtered, and structurally transformed.

The primary source selected for this purpose is the *TBO Hotels Dataset*, hosted on the Kaggle platform¹. This repository offers an extensive global catalog of accommodation properties, providing attributes such as property names, star ratings, geographical coordinates, physical addresses, contact details, and narrative overviews.

5.1.1 Geographical Filtering and Selection

The original Kaggle repository contains hundreds of thousands of lodging records scattered worldwide, with an initial dimension of over 2GB. Retaining the entire dataset would have introduced severe structural imbalance into our application: preliminary data exploration revealed that several European capitals lacked a statistically sufficient sample size to support our platform’s core features (such as spatial neighborhood routing and dynamic travel package retrieval).

To resolve this discrepancy, a strict geographical pruning was executed. We filtered the raw data to retain accommodations located exclusively within a carefully selected network of 19 European capitals that guaranteed balanced data density across all desired metrics. The final cities were reduced to Amsterdam (Netherlands), Berlin (Germany), Bern (Switzerland), Copenhagen (Denmark), Dublin (Ireland), London (United Kingdom), Oslo (Norway), Paris (France), Stockholm (Sweden), Athens (Greece), Madrid (Spain), Rome (Italy), Tirana (Albania), Bratislava (Slovakia), Budapest (Hungary), Moscow (Russia), Prague (Czech Republic), Sofia (Bulgaria), and Vienna (Austria).

¹<https://www.kaggle.com/datasets/raj713335/tbo-hotels-dataset>

Properties located outside these 19 capitals were discarded, establishing a geographically coherent catalog.

5.1.2 Data Preprocessing and Schema Transformation

Once the geographical subset was isolated, the raw CSV records underwent a series of pre-processing activities to map the tabular format into a JSON schema. To do this, it was important to follow a pipeline of many steps.

First of all, irrelevant metadata from the original source were stripped, since the application did not need it. The remaining fields were logically aggregated into nested sub-documents (such as `hotel_info`, `address`, and `contacts`) to improve semantic clarity.

The second step in this transformation was very important for our needs: geographical coordinates, originally exported as separate latitude and longitude floating-point columns, were transformed into a standard GeoJSON object, to make them compatible with geospatial queries both on MongoDB and Neo4J.

Finally, to support the package scoring algorithms, a realistic estimated average nightly rate (`price_average`) was calculated and integrated into each document, giving a base price to every city according to the cost of living there and weighed it by a multiplier based on the number of stars of the hotel.

At the end of the transformation pipeline, the generated JSON documents achieved the standardized structure illustrated.

```

1 {
2   "hotel_info": {
3     "name": "De Paris Hotel",
4     "stars": 4,
5     "website": "https://www.booking.com/hotel/al/de-paris.html"
6   },
7   "location": {
8     "type": "Point",
9     "coordinates": [ 19.81665, 41.32213 ]
10  },
11  "address": {
12    "city": "Tirana",
13    "country": "Albania",
14    "street": "Nr. 7 Brigada Viii Street Tirane ",
15    "zip": "1000"
16  },
17  "amenities": [
18    "Free", "self", "parking", "Restaurant",
19    "Pets", "allowed", "Room", "service",
20    "Meeting", "rooms", "Bar/lounge", ...
21  ],
22  "contacts": {
23    "phone": "00355 4226 5009",
24    "fax": "42268822"
25  },
26  "description": "Hotel de Paris is a charming boutique hotel...",
27  "price_average": 152.53,
28  "_id": {
29    "$oid": "286a246ec164432cc055cef4"
30  }
31 }

```

5.2 Flight Data Acquisition and Synthetic Generation

To support the dynamic packaging mechanics of the platform, an extensive aerial transportation catalog was engineered.

The source of this repository was derived from the open-source *OpenFlights*² database. Specifically, we extracted baseline routes, capturing original airlines and directional connections expressed via standard 3-letter IATA (or 4-letter ICAO) airport codes.

Consistently with the constraints enforced during the hotel data selection, the flight dataset was subjected to topological pruning. We filtered the raw routes to retain exclusively the aerial pathways connecting our 19 designated European capitals.

However, relying strictly on historical static routes was problematic, since the flights were missing temporal scheduling. We needed to ensure that users planning trips months in advance would never encounter dead ends.

For every valid origin-destination pair within our 19-city network, based on the routes we had from the original source, synthetic outbound and return flights' schedules were built to guarantee a strict minimum density of at least 2 outbound and 2 return flights per month stretching continuously from the present baseline up to the end of the year 2027.

Moreover, it was necessary to enrich the routes with realistic pricing: for the recommendation algorithm of the application, plausible baseline ticket prices were calculated.

5.3 Points of Interest (POI) Extraction

To obtain Points of Interests for each European capital and for each interest category we designed, we extracted urban attractions directly from *OpenStreetMap* via the *Overpass Turbo*³ API.

Data retrieval was executed per-city using Overpass queries. The extraction targeted attractions that matched our application's six core interest categories: *culture*, *relax*, *shopping*, *nightlife*, *nature*, and *food*.

The Overpass JSON output underwent a preprocessing step: one action was for the purpose of collapsing complex polygons into [longitude, latitude] arrays; finally, the structure of the output JSON document was very unorganized, so every key-value pair that was not needed in the final document was encapsulated into a metadata sub-document.

The processed JSON documents were finally imported into **Neo4j** as :POI nodes.

```

1 {
2   "name": "European long distance path E8 - part Slovakia",
3   "city": "Bratislava",
4   "category": "nature",
5   "location": {
6     "type": "Point",
7     "coordinates": [ 19.3791025, 48.7780283 ]
8   },
9   "address": { "street": "N/A", "postcode": "N/A" },

```

²<https://openflights.org/data>

³<https://overpass-turbo.eu/>

```

10  "metadata": {
11    "complete": "yes",
12    "network": "iwn",
13    "route": "hiking",
14    "website": "https://www.era-ewv-ferp.org/...",
15    "wikidata": "Q106129138"
16  }
17 }

```

5.4 Travel Packages Synthetic Generation

The core entities of the platform are represented by the *Travel Packages*, which act as unified, ready-to-book itineraries. We designed a data synthesis script developed in Python using the Faker library.

This generation pipeline links and synthesizes records from our previously cleaned repositories (*hotels*, *POI* and our synthetic *reviews*).

The Python script constructed each package using an embedding strategy. In fact, the script generated contextual text hints (*clues*) and titles tailored to the target destination, seasons and interests. Moreover, a few details of the hotel were embedded, such as *hotel_info*, *location* and *address*, and given the price of the hotel a *price* field was generated for each travel package.

Exactly 5 POIs were sampled per package, which categories are adherent with the features of the travel package. Finally, from the reviews collections a *rating summary* was generated and the last three reviews were embedded inside the package.

5.5 User Profiles Synthetic Generation

The user base of the platform was generated through Python scripts. In particular, the **Faker** library was employed to synthesize realistic baseline identities, including names, national phone numbers, nationalities, birth dates, and identity documents (such as passports).

Each user profile was enriched with a highly denormalized transactional history: the baseline attributes were encapsulated into the sub-documents (**credentials**, **personal_info**, and **identity_documents**); a **cart** object was attached to selected profiles, embedding eventual travel packages alongside its outbound and return flight summaries; finally, the **orders** array was synthesized and embedded inside the user profile, where each one denormalizes the purchased itinerary details, thematic clues, surprise flags, and partial embedded flight schedules.

5.6 Questionnaires and Reviews Synthetic Data

To capture feedback and test the application's recommendation algorithms, two distinct user-generated datasets were synthetically generated via Python scripts: *Questionnaires* and *Reviews*.

First of all, concerning the questionnaires, each one captures an explicit onboarding session, grouping user constraints (budget limits, travel style, preferred season, and age target) into a **preferences** sub-document. Crucially, the schema denormalizes the algorithm's response

directly within a `generated_suggestions` object. This nested structure records the timestamp, the top three ranked packages alongside their computed `match_score`, and the final `user_selection`.

Finally, feedback was modeled through a dedicated `reviews` collection.

5.6.1 Dataset Statistics

At the end of the data acquisition, pre-processing, and synthetic generation workflows, these repositories contain the personal credentials and interaction history of our registered customer base, alongside the comprehensive catalog of accommodations, flight schedules, points of interests, and travel packages.

In terms of physical volume, the generated dataset comprises seven primary files reaching a cumulative uncompressed footprint of approximately **324 MB**.

Chapter 6

DB Design

Given the large volume of data the system must handle, the choice of database architecture is crucial to ensuring high performance and service reliability. To achieve optimal efficiency and satisfy the specific functional needs of the application, we decided to distribute our data across two different types of large-scale databases, each chosen for its specific strengths: MongoDB and Neo4j.

6.1 Document DB - MongoDB

MongoDB was adopted as the main data store for the application's foundational entities: Flights, Travel Packages, Users, Reviews, and Questionnaires. The platform relies heavily on MongoDB's flexible, schema-free nature to model intricate and multi-structured data.

6.1.1 Collections

The MongoDB database is structured in the following collections:

- Flights
- Travel Packages
- Users
- Questionnaires
- Reviews

Flights

```
1 {  
2   "_id": {  
3     "$oid": "38b1843646fb86b883f81a37"  
4   },  
5   "route_id": "2B-6311",  
6   "airline": "2B",  
7   "departure": {  
8     "airport_iata": "KZN",  
9     "city": "Kazan",  
10    "at": "2027-05-11T20:48:00.000+00:00"  
11  },  
12  "arrival": {  
13    "airport_iata": "DME",  
14    "city": "Moscow",  
15    "at": "2027-05-11T22:11:00.000+00:00"
```

```

16   },
17   "price": 110.05,
18   "duration_minutes": 83
19 }

```

Inside the `Flights` collection, each document models a self-contained route connecting two cities within the platform's network.

Alongside basic identifiers such as the (`airline`) and the specific `route_id`, the schema structures spatial and scheduling data into two embedded sub-documents: `departure` and `arrival`. Each nested object encapsulates the airport IATA code, the host city name, and the exact timestamp of departure and arrival.

Finally, storing scalar values for ticket `price` and `duration_minutes` directly at the root level allows the application's recommendation engine to execute instantaneous budget filtering and itinerary sorting.

Travel Packages

```

1  {
2    "_id": { "$oid": "7ba504c7c01a4d30557f6fac" },
3    "title": "The Traditions of Dublin",
4    "city": "Dublin",
5    "clues": [ "Literary city", "Rich in history and art", ... ],
6    "features": {
7      "preferred_season": "summer",
8      "travel_style": "standard",
9      "interests": [ "culture", "relax", "food" ]
10   },
11   "activities": [
12     {
13       "name": "Charles Stewart Parnell Monument",
14       "category": "Culture",
15       "location": { "type": "Point", "coordinates": [ -6.2614772, 53.352594 ] }
16     }, ...
17   ],
18   "hotel": {
19     "hotel_info": { "name": "Mount Street Luxury Suite", "stars": 4, ... },
20     "location": { "type": "Point", "coordinates": [ -6.24343, 53.33861 ] },
21     "address": { "city": "Dublin", "country": "Ireland(Republic of)", ... }
22   },
23   "rating_summary": { "average": 4.02, "total_reviews": 59 },
24   "price": 512.36,
25   "latest_reviews": [
26     {
27       "user_name": "Guido V.",
28       "rating": 3,
29       "comment": "Nella media, un buon weekend.",
30       "created_at": { "$date": "2025-10-17T17:37:00Z" }
31     }, ...
32   ]
33 }

```

Within the `Travel Packages` collection, each document represents a fully aggregated, read-optimized itinerary engineered to serve best client-side browsing requests. A document of this collection is composed by *title*, *city*, an array of *clues* (useful for the surprise option), *features* (preferred_season, travel_style and an array of interests), an array of embedded *activities*, a

few embedded fields for the *hotel*, a *rating_summary* (composed of average rating and total number of reviews), the *price* of the package template and an array of three embedded reviews (latest_reviews).

In particular, since we do not plan on storing POIs on MongoDB but rather on Neo4j, the document embeds an array of exactly five attractions, preserving only their display names, interest categories, and 2dsphere-compatible GeoJSON coordinates.

Moreover, the hotel profile is completely denormalized, embedded directly into the package structure, granting immediate access to name, property stars, website, geolocation and the address information.

To support instantaneous UI access to the last reviews in the Package Details page, an overall review counter is maintained alongside an embedded subset of the three most recent feedback entries, each containing the username, rating, comment and timestamp of the review and user who wrote it. This section, relative to the rating summary and latest reviews, will need to be updated every time a new review gets written about that specific package, to maintain data consistency across collections.

Finally, from an architectural standpoint, this deep embedding strategy is mathematically safe against document bloat. Because the nested arrays are strictly bounded by application-level mechanics (fixed at 5 activities and a maximum of 3 reviews), and given that the travel package templates will not change regarding the dimension of said arrays, the overall physical volume of a Travel Package document remains deterministic and orders of magnitude below MongoDB's 16MB limit.

Users

```

1 {
2   "_id": { "$oid": "26a1977725c3a3c25c4e5bd3" },
3   "role": "customer",
4   "credentials": {
5     "email": "michaela.giron990@yahoo.com",
6     "password_hash": "
7     $2b$12$e0ybQvTWXGdTodFfNzs4x3jQbYTgWhu3ae9hsM5nqeQdyLVlfdw.t"
8   },
9   "personal_info": {
10     "first_name": "Micaela", "last_name": "Giron",
11     "phone": "+34973588554", "nationality": "Spain",
12     "birth_date": "1983-07-22T00:00:00.000+00:00"
13   },
14   "identity_documents": [
15     { "type": "passport", "number": "SL1895598", ... }
16   ],
17   "cart": {
18     "items": [ { "package_id": "efc7b4...", "name": "Echoes of London", ... } ],
19     "flight_summary": { "outbound": { ... }, "return": { ... } },
20     "total_estimated": 765.79
21   },
22   "orders": [
23     {
24       "order_id": "6a215c...", "package_id": "fa9e53...",
25       "name": "The Secrets of London", "order_status": "confirmed",
26       "clues": [ "Ride a red bus", ... ],
27       "flight_summary": { "outbound": { ... }, "return": { ... } },
28       "price": 1155.21
29     }
30   ]
31 }

```

```

28     }, ...
29   ]
30 }

```

Inside the `Users` collection, each document models a registered customer profile enriched with its transactional lifecycle. Alongside core access authorization (`role`), baseline identity attributes are logically segregated into dedicated nested sub-documents (`credentials`, `personal_info`, and `identity_documents`) to facilitate targeted updates.

Crucially, authentication security is guaranteed at the schema level by persisting password strings processed via the `bcrypt` hashing algorithm.

The document relies on a hybrid data modeling strategy, since transactional attributes inside the `cart` and `orders` structures are engineered combining partial embedding and document linking. In fact, both the `cart.items` and the `orders` arrays avoid embedding heavy domain entities (such as complete hotel profiles or activity arrays). Instead, the schema denormalizes purely a lightweight subset: package titles, destinations, `clues`, surprise flags, and prices.

However, it's necessary to maintain referential integrity and prevent severe document bloat, so some external pointers must be preserved. Each staged cart item and confirmed order explicitly retains the target `package_id`, alongside distinct `flight_id` references nested within the outbound and return `flight_summary` sub-documents. Whenever a customer demands deep inspection (e.g., clicking "View Flight Details"), the backend can seek directly on the external `flights` or `packages` collections.

While continuously appending confirmed transactions inherently increases document size, the risk of reaching MongoDB's strict 16MB physical limit is neutralized by this design. Because the embedded payloads are stripped of heavy sub-documents, an individual historical order consumes less than 0.6 KB. Even in an unlikely scenario of a user accumulating 500 confirmed lifetime trips, the total transactional overhead would barely reach 0.3 MB, securing high read performance.

Questionnaires

```

1  {
2    "_id": { "$oid": "72c82ceb9c4450ae0199294a" },
3    "user_id": { "$oid": "f091f9f98a71352d000f1d4b" },
4    "submitted_at": "2024-06-13T19:36:00.000+00:00",
5    "preferences": {
6      "preferred_season": "winter", "travel_style": "budget",
7      "departure_city": "Dresden", "budget_limit": 500,
8      "interests": [ "food" ], "is_surprise": false, "age_range": "66-75"
9    },
10   "generated_suggestions": {
11     "suggested_at": "2024-06-13T19:41:00.000+00:00",
12     "packages": [
13       { "package_id": "4a3d92...", "match_score": 90, "name": "Artistic Soul
14         of Moscow" },
15       { "package_id": "0edba2...", "match_score": 75, "name": "The Secrets of
16         Moscow" }
17     ]
18   },
19   "user_selection": { "$oid": "4a3d9200d1aa70eecf6e0ffd" }
20 }

```

Within the Questionnaires collection, each document models the transition from raw user answers to questionnaire’s questions, to algorithmic itinerary generation. All input criteria submitted by the user during the questionnaire flow, such as `departure_city`, the array of `interests`, the flag for surprise generation `is_surprise`, `budget_limit`, `travel_style`, `preferred_season`, and `age_range`, are fully embedded into a cohesive `preferences` sub-document.

To record the recommendation algorithm’s output without bloating the BSON structure, the document embeds a lightweight projection of the top recommended itineraries. Instead of encapsulating full `TravelPackage` entities (which would redundantly duplicate hotel and attraction details), the `packages` array denormalizes strictly the target `package_id`, the computed `match_score`, and the display `name`, allowing for a partial embedding structure. This means that the `generated_suggestions` field will be empty until the user completes the related questionnaire, saving not only the suggestions but also the timestamp.

Finally, the document preserves explicit relational pointers, actuating a document linking method for the field `user_selection`, since it’s necessary for the user to visualize the full package details once selected. The root `user_id` links the questionnaire session to the `Users` repository, while `user_selection` stores the exact `ObjectId` of the final itinerary chosen by the customer.

From a storage perspective, this hybrid schema is strictly deterministic.

Because the `generated_suggestions.packages` array is bounded by the recommendation engine’s projection limit (fixed at 3 suggestions), an individual Questionnaire document maintains a negligible physical footprint, entirely eliminating the risk of unbounded document growth.

Reviews

```

1 {
2   "_id": { "$oid": "6a3d4845ac2b2392291b8244" },
3   "user_id": { "$oid": "f091f9f98a71352d000f1d4b" },
4   "package_id": { "$oid": "35db403defe204a424b9765e" },
5   "user_name": "Fortunata M.",
6   "package_title": "Masterpieces of London",
7   "rating": 5,
8   "comment": "An unforgettable experience in London...",
9   "created_at": "2024-11-05T16:55:00.000+00:00",
10  "is_surprise": true
11 }
```

Inside the Reviews collection, each document captures an immutable post-trip evaluation submitted by a registered user for a specific travel.

The schema is heavily engineered around read-time denormalization: alongside foundational relational pointers (`user_id` and `package_id`), the document explicitly duplicates UI projection attributes such as the author’s display `user_name` and the target `package_title`. This partial embedding strategy allows profile feedback tabs and package review feeds to display the review instantaneously.

From a volume perspective, this structure maintains an extremely lightweight physical footprint, guaranteeing horizontal scalability even under high-write workloads.

6.2 GraphDB - Neo4j

While MongoDB provides an optimal solution for storing semi-structured, document-based transactional data, the relational complexity of travel itineraries, involving user preferences, thematic categorization, and spatial proximity, demands a dedicated graph-oriented approach. Neo4j was therefore adopted as the graph database to manage the application's relationships, enabling efficient traversal across users, travel packages, and points of interest.

The integration of Neo4j serves three architectural objectives:

- **For You section:** By traversing the graph through past purchases and review history, the engine identifies latent similarities between user profiles and package attributes, enabling the generation of personalized travel suggestions.
- **Popular section:** The graph structure allows for high-performance retrieval of top-ranked travel packages based on various metrics, such as aggregate review scores and purchase volume.
- **Proximity Analysis for Custom POIs:** Neo4j enables complex spatial queries, such as the discovery of *custom POIs* based on their distance from an hotel's location. This ensures that the suggestions of nearby activities to do for the user remains both logically coherent and geographically relevant to the user's specific itinerary.

6.2.1 Graph Nodes

To support these features, the following entities have been modeled as nodes within the graph database:

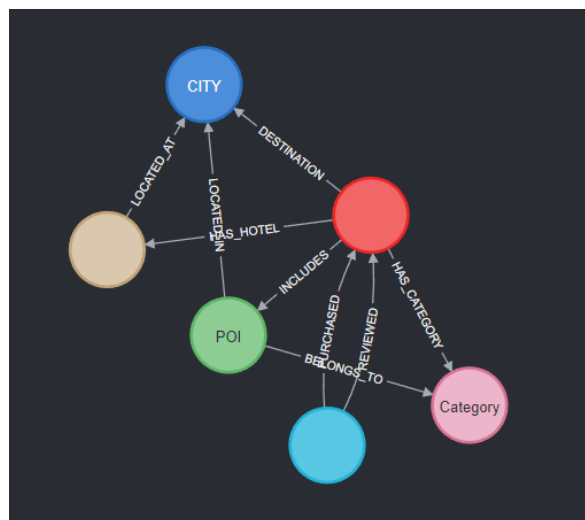
- **Travel Package:** represents the homonym entity inside the Mongo database. Each node stores its unique identifier along with basic information retrieved from MongoDB, specifically the `mongo_id`, title, average rating and price.
- **User:** represents the User entity. Each node stores its unique identifier along with basic information retrieved from MongoDB, such as the `mongo_id`, first name, last name, birth date and role.
- **POI:** represents the Points of Interest entity. Each node stores its unique identifier along with information retrieved from MongoDB, such as the `mongo_id`, name and location.
- **Hotel:** such as the POI node, each node of the Hotel entity stores its unique identifier and information such as the `mongo_id`, name and location.
- **Category:** represents the 6 main categories of interests in which the travel packages are divided. These are consequently 6 nodes: Culture, Relax, Nightlife, Nature, Shopping and Food, only storing their unique identifiers and their name.
- **City:** represents the 19 cities involved in our application. Each node stores their unique identifier, the name of the city and the name of the country.

6.2.2 Graph Relationships

The relationships involved in the graph database are the following:

- **REVIEWED:** connects a User with a Travel Package, representing a user that wrote a review on that specific package. It has 2 attributes: rating and timestamp.

- **PURCHASED**: connects a User with a Travel Package, representing a user that bought that specific package. It has the attribute timestamp.
- **BELONGS_TO**: connects a POI with a Category, representing a POI that belongs to that specific category.
- **HAS_CATEGORY**: connects a Travel Package with a Category, representing a Travel Package that belongs to that specific category.
- **INCLUDES**: connects a Travel Package with a POI, representing a Travel Package containing those specific POIs.
- **DESTINATION**: connects a Travel Package with a City, representing a Travel Package which activities are located in that specific city.
- **LOCATED_IN**: connects a POI with a City, representing a POI located in that specific City.
- **LOCATED_AT**: connects a Hotel with a City, representing a Hotel located in that specific City.
- **HAS_HOTEL**: connects a Travel Package with a Hotel, representing a Travel Package that has that specific hotel.



Chapter 7

Implementation

In this chapter, the detailed implementation of the software architecture and the backend infrastructural code of the application will be discussed in detail. The Spring Boot framework was used to implement the application in order to build a robust, scalable, and cohesive project.

The purpose of this section is to discuss and explain the choices regarding the application architecture and the specific role of the various components.

Architecturally, the application follows the Spring Boot MVC (Model View Controller) design pattern. To maintain a rigorous separation of concerns, enforce clean boundaries, and encourage modular code reusability, the entire codebase is organized into distinct packages. Each package covers a single operational role identified during the system analysis phase, which completely isolates data persistence, business logic processing, and client-side communication.

7.1 Spring Framework

This project is built upon the Spring Boot ecosystem, which was selected due to its leading role in creating scalable and distributed software solutions. It simplifies the configuration usually associated with Spring applications by offering automated starters and dependency management. It contains an embedded Apache Tomcat servlet container, which greatly accelerates local testing and cloud-native deployment cycles.

Most importantly, the data access layers are abstracted using the **Spring Data** umbrella framework, with the application mainly relying on **Spring Data MongoDB** and **Spring Data Neo4j**. This abstraction allows the application to manage data through very intuitive object-relational/object-document mapping (ORM/ODM) annotations.

7.2 Config

The `config` package contains the configuration class required to manage external services within the application. Specifically, the following class is implemented:

- **Neo4jIndexConfig**: This class acts as an automated database administrator at the application layer. On system startup, it interacts directly with the graph database using Spring's native `Neo4jClient`. It verifies and automatically executes native Cypher queries to guarantee the presence of indexes in the graph database. In particular, it ensures that indexes are created on the `id` attribute of the `User` and `Travel Package` nodes, and on the `birth date` attribute of the `User` node. This indexing is crucial to unlock sub-millisecond execution time for the system's recommendation queries, pre-

venting full-graph scans when computing suggested travel packages based on age or interests.

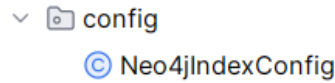


Figure 7.1: Config package folder.

7.3 Event

The **event** package is responsible for implementing asynchronous task execution within the system. In a polyglot system, writing data to multiple distinct database synchronously introduces severe latency overhead and exposes the application to a potential synchronization cascade due to write-failure. Because of this, synchronization tasks or secondary database mutations are separated from the primary transactional thread and loaded to an asynchronous background processing subsystem.

- **Task:** This class is as a domain-specific enumeration (**enum**) that categorizes all permissible background operations within the application lifecycle. It also allows to track down and discriminate among events caused by state changes of the main database, including:
 - **SYNC_NEW_USER:** Occurs when a customer registers, indicating the need to instantiate a corresponding user node in the graph database.
 - **DELETE_USER_NEO4J:** Occurs when an account is deleted, enforcing cascading detachment of graph relations.
 - **ADD_PURCHASED_RELATION:** Occurs at cart checkout, signaling the graph database to draw a relationship between a user node and a travel package node.
 - **ADD_REVIEWED_RELATION:** Occurs when a review is submitted, signaling the graph database to draw a relationship between a user node and a travel package node.
- **TaskToDo:** This class acts as a task manager, representing a specific unit of work that needs to be executed. It encapsulates a specific **Task** type along with its corresponding data payload. To handle heterogeneous event structures, it implements multiple constructors. This configuration allows a task to be instantiated depending on the specific operation: from a simple string identifier for quick user deletions, to fully detailed payloads carrying ratings, ISO timestamps, or complex multi-structured domain objects.

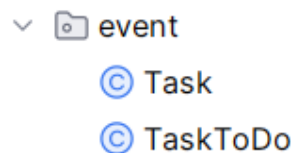


Figure 7.2: Event package folder.

7.4 Model

The **model** package represents the structural definition of the platform’s core entities. The domain models are split into two separated sub-packages, each tailored to maximize the native architectural benefits of its database engine.

7.4.1 MongoDB Models

Flight

```

1  /**
2   * MongoDB document entity representing a flight route.
3   * Utilizes a compound index on departure city, arrival city, and departure
4   * time
5   * to heavily optimize flight search queries.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 @Document(collection = "flights")
11 @CompoundIndexes({
12     @CompoundIndex(name = "idx_flight_search", def = "{ 'departure.city': 1, '
13     arrival.city': 1, 'departure.at': 1 }")
14 })
15 public class Flight {
16     @Id
17     private ObjectId id;
18     @Field("route_id")
19     private String routeId;
20     private String airline;
21     private double price;
22     @Field("duration_minutes")
23     private int durationMinutes;
24     private FlightNode departure;
25     private FlightNode arrival;

```

Listing 7.1: Flight Document Implementation

```

1  /**
2   * Embedded document representing a specific flight details.
3   * Encapsulates airport information, city, and timing.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  public class FlightDetail {
9     @Field("airport_iata")
10     private String airportIata;
11     private String city;
12     private Instant at;
13 }

```

Listing 7.2: Embedded Flight Document Implementation

Questionnaire

```

1  /**
2   * MongoDB document entity representing a user's submitted travel
3   * questionnaire.
4   * Stores the user's travel preferences and the subsequent system-generated
5   * package suggestions.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 @Document(collection = "questionnaires")
11 public class Questionnaire {

```

```

10     @Id
11     private String id;
12     @Field(name = "user_id", targetType = FieldType.OBJECT_ID)
13     private String userId;
14     @Field("submitted_at")
15     private Instant submittedAt;
16     private QuestFeature preferences;
17     @Field("generated_suggestions")
18     private Suggestion generatedSuggestions;
19 }

```

Listing 7.3: Questionnaire Document Implementation

```

1  /**
2   * Embedded document capturing the exact travel preferences submitted by the
3   * user.
4   * Acts as the input parameter set for the recommendation matching algorithm.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  public class QuestFeature {
10     @Field("preferred_season")
11     private String preferredSeason;
12     @Field("travel_style")
13     private String travelStyle;
14     private List<String> interests;
15     @Field("departure_city")
16     private String departureCity;
17     @Field("budget_limit")
18     private double budgetLimit;
19     @Field("is_surprise")
20     private boolean isSurprise;
21     @Field("age_range")
22     private String ageRange;
23 }

```

Listing 7.4: Embedded Questionnaire Document Implementation

```

1  /**
2   * Embedded document representing the output of the recommendation engine.
3   * Contains the list of suggested packages and records the user's final
4   * selection.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  public class Suggestion {
10     @Field("suggested_at")
11     private Instant suggestedAt;
12     private List<SuggPackage> packages;
13     @Field("user_selection")
14     @JsonSerialize(using = ToStringSerializer.class)
15     private ObjectId userSelection;
16 }

```

Listing 7.5: Embedded Questionnaire Document Implementation

```

1  /**

```

```

2  * Embedded document representing an individual travel package suggested to
    the user.
3  * Employs the @Transient annotation for dynamic fields (like prices and
    specific flights)
4  * that are computed at runtime based on real-time availability, rather than
    stored persistently in the DB.
5  */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  public class SuggPackage {
10     @Field("package_id")
11     @JsonSerialize(using = ToStringSerializer.class)
12     private ObjectId packageId;
13     @Field("match_score")
14     private double matchScore;
15     private String name;
16     @Transient
17     private double price;
18     @Transient
19     private double basePrice;
20     @Transient
21     private int durationDays;
22     @Transient
23     @JsonSerialize(using = ToStringSerializer.class)
24     private ObjectId outboundFlightId;
25     @Transient
26     @JsonSerialize(using = ToStringSerializer.class)
27     private ObjectId returnFlightId;
28     @Transient
29     private Instant departureDate;
30     @Transient
31     private Instant returnDate;
32 }

```

Listing 7.6: Embedded Questionnaire Document Implementation

Review

```

1  /**
2  * MongoDB document entity representing a user review for a travel package.
3  * Includes a specific index on {@code package_id} to optimize queries
    fetching
4  * all reviews associated with a single travel package.
5  */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  @Document(collection = "reviews")
10 public class Review {
11     @Id
12     private String id;
13     @Field(name = "user_id", targetType = FieldType.OBJECT_ID)
14     private String userId;
15     @Field(name = "package_id", targetType = FieldType.OBJECT_ID)
16     @Indexed(name = "package_id_idx")
17     private String packageId;
18     @Field("user_name")
19     private String userName;
20     @Field("package_title")
21     private String packageTitle;
22     private double rating;

```

```

23     private String comment;
24     @Field("created_at")
25     private Instant createdAt;
26     @Field("is_surprise")
27     private boolean isSurprise;
28 }

```

Listing 7.7: Review Document Implementation

Travel Package

```

1  /**
2   * Core MongoDB document entity representing a complete travel package
3   * offering.
4   * Aggregates all package details including features, hotel details, POIs (
5   * activities),
6   * and a denormalized summary of the latest reviews to speed up read
7   * operations.
8   * Indexed by city for rapid destination-based filtering.
9   */
10 @Data
11 @AllArgsConstructor
12 @NoArgsConstructor
13 @Document(collection = "travel_packages")
14 public class TravelPackage {
15     @Id
16     private String id;
17     private String title;
18     @Indexed(name = "idx_city")
19     private String city;
20     private List<String> clues;
21     private PackageFeature features;
22     private List<PointOfInterest> activities;
23     private HotelDetail hotel;
24     @Field("rating_summary")
25     private Rating ratingSummary;
26     private double price;
27     @Field("latest_reviews")
28     private List<ReviewPackage> latestReviews;
29 }

```

Listing 7.8: Travel Package Document Implementation

```

1  /**
2   * Embedded document representing a physical address.
3   * Nested within hotel details to provide structured location data.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  public class Address {
9      private String city;
10     private String country;
11     private String street;
12     private String zip;
13 }

```

Listing 7.9: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document aggregating hotel information.
3   */
4  @Data
5  @AllArgsConstructor
6  @NoArgsConstructor
7  public class HotelDetail {
8      @Field("hotel_info")
9      private HotelInfo hotelInfo;
10     private GeoJsonPoint location;
11     private Address address;
12 }

```

Listing 7.10: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document containing the core metadata of a hotel.
3   */
4  @Data
5  @AllArgsConstructor
6  @NoArgsConstructor
7  public class HotelInfo {
8      private String name;
9      private int stars;
10     private String website;
11 }

```

Listing 7.11: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document defining the characteristics of a travel package.
3   */
4  @Data
5  @AllArgsConstructor
6  @NoArgsConstructor
7  public class PackageFeature {
8      @Field("preferred_season")
9      private String preferredSeason;
10     @Field("travel_style")
11     private String travelStyle;
12     private List<String> interests;
13 }

```

Listing 7.12: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document representing a localized activity or attraction within a
3   * package.
4   * The @JsonIgnoreProperties annotation ensures clean JSON serialization
5   * by hiding redundant internal coordinate fields.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 public class PointOfInterest {
11     private String name;
12     private String category;
13     @JsonIgnoreProperties({"x", "y"})
14     private GeoJsonPoint location;

```

14 }

Listing 7.13: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document storing aggregated rating metrics.
3   * Caches the average score and total review count directly inside the
4   * package document
5   * to prevent expensive recalculations during high-volume read operations.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 public class Rating {
11     private Double average;
12     @Field("total_reviews")
13     private int totalReviews;
14 }

```

Listing 7.14: Embedded Travel Package Document Implementation

```

1  /**
2   * Embedded document containing a user review.
3   * Store a subset of the latest reviews directly inside the travel package.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  public class ReviewPackage {
9     @Field("user_name")
10     private String userName;
11     @Field("package_title")
12     private String packageTitle;
13     private double rating;
14     private String comment;
15     @Field("created_at")
16     private Instant createdAt;
17 }

```

Listing 7.15: Embedded Travel Package Document Implementation

User

```

1  /**
2   * MongoDB document entity representing a registered user in the system.
3   * Aggregates personal information, identity documents, the current shopping
4   * cart,
5   * and the user's entire order history all in a single document.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 @Document(collection = "users")
11 public class User {
12     @Id
13     private String id;
14     private String role;
15     private Credentials credentials;
16     @Field("personal_info")
17     private PersonalInfo personalInfo;

```

```

17     @Field("identity_documents")
18     private List<IdentityDocument> identityDocuments;
19     private Cart cart;
20     private List<Order> orders;
21 }

```

Listing 7.16: User Document Implementation

```

1  /**
2   * Embedded document representing a user's shopping cart.
3   * Aggregates pending items and calculates the running total before checkout.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  public class Cart {
9      private List<Item> items = new ArrayList<>();
10     @Field("total_estimated")
11     private double totalEstimated;
12 }

```

Listing 7.17: Embedded User Document Implementation

```

1  /**
2   * Embedded document storing the user's authentication credentials.
3   * Secures the application by isolating the hashed password from general
4   * profile data.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  public class Credentials {
10     private String email;
11     @Field("password_hash")
12     private String password;
13 }

```

Listing 7.18: Embedded User Document Implementation

```

1  /**
2   * Embedded document representing a flight instance within a user's cart or
3   * order.
4   */
5  @Data
6  public class EmbeddedFlight {
7      @Field("flight_id")
8      @JsonSerialize(using = ToStringSerializer.class)
9      private ObjectId flightId;
10     @Field("departure_city")
11     private String departureCity;
12     @Field("arrival_city")
13     private String arrivalCity;
14     @Field("departure_date")
15     private Instant departureDate;
16     @Field("arrival_date")
17     private Instant arrivalDate;
18 }

```

Listing 7.19: Embedded User Document Implementation

```

1  /**
2   * Embedded document summarizing the complete flight itinerary (outbound and
3   *   return).
4   * Used within Items and Orders to link specific travel packages to actual
5   *   flight selections.
6   */
7  @Data
8  public class EmbeddedFlightSummary {
9      @Field("outbound")
10     private EmbeddedFlight outboundFlight;
11     @Field("return")
12     private EmbeddedFlight returnFlight;
13 }

```

Listing 7.20: Embedded User Document Implementation

```

1  /**
2   * Embedded document containing identity verification details.
3   * Kept strictly within the User document to ensure compliance and privacy.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  public class IdentityDocument {
9      private String type;
10     private String number;
11     @Field("expiry_date")
12     private LocalDate expiryDate;
13 }

```

Listing 7.21: Embedded User Document Implementation

```

1  /**
2   * Embedded document representing an individual travel package currently in
3   *   the user's cart.
4   */
5  @Data
6  public class Item {
7      @Field("package_id")
8      @JsonSerialize(using = ToStringSerializer.class)
9      private ObjectId packageId;
10     private String name;
11     private String destination;
12     @Field("added_at")
13     private Instant addedAt;
14     @Field("is_surprise")
15     private boolean isSurprise;
16     private double price;
17     @Field("flight_summary")
18     private EmbeddedFlightSummary flightSummary;
19     @Transient
20     private String censoredTitle;
21     private List<String> clues;
22 }

```

Listing 7.22: Embedded User Document Implementation

```

1  /**
2   * Embedded document representing a finalized purchase order.

```

```

3  * Serves as an immutable historical record of a transaction, capturing the
4  * exact price, flights,
5  * and custom POIs in the user's history.
6  */
7  @Data
8  public class Order {
9      @Field("order_id")
10     @JsonSerialize(using = ToStringSerializer.class)
11     private ObjectId orderId;
12     @Field("package_id")
13     @JsonSerialize(using = ToStringSerializer.class)
14     private ObjectId packageId;
15     private String name;
16     private String destination;
17     private List<String> clues;
18     @Field("purchase_date")
19     private Instant purchaseDate;
20     @Field("order_status")
21     private String orderStatus;
22     @Field("is_surprise")
23     private boolean isSurprise;
24     private double price;
25     @Field("flight_summary")
26     private EmbeddedFlightSummary flightSummary;
27     @Field("custom_pois")
28     private List<CustomPoiDTO> customPois;
29     private String censoredTitle;
30 }

```

Listing 7.23: Embedded User Document Implementation

```

1  /**
2   * Embedded document storing basic demographic and contact information.
3   * Calculates the user's current age dynamically via a @Transient field,
4   * ensuring the age
5   * is always accurate relative to the current system date without needing
6   * background DB updates.
7   */
8  @Data
9  @AllArgsConstructor
10 @NoArgsConstructor
11 public class PersonalInfo {
12     @Field("first_name")
13     private String firstName;
14     @Field("last_name")
15     private String lastName;
16     private String userName;
17     private String phone;
18     private String nationality;
19     @Field("birth_date")
20     private LocalDate dateOfBirth;
21     @Transient
22     private Integer age;
23
24     public Integer getAge() {
25         if (this.dateOfBirth != null) {
26             return Period.between(this.dateOfBirth, LocalDate.now()).getYears();
27         }
28         return null;
29     }
30 }

```

Listing 7.24: Embedded User Document Implementation

Note: To go along with MongoDB’s document-based nature, several embedded sub-documents were modeled. These entities utilize specific annotations like `@Transient` to manage data computed at runtime (e.g. censored titles) and `GeoJsonPoint` to support spatial mapping.

7.4.2 Neo4j Graph Models

The Neo4j models represent the structural nodes and semantic relationships mapping the graph database.

Category Node

```

1  /**
2   * Neo4j Node entity representing a generic classification category
3   */
4  @Data
5  @AllArgsConstructor
6  @NoArgsConstructor
7  @Node("Category")
8  public class CategoryNode {
9      @Id
10     private String name;
11 }

```

Listing 7.25: Category Node Implementation

City Node

```

1  /**
2   * Neo4j Node entity representing a geographical City.
3   */
4  @Data
5  @AllArgsConstructor
6  @NoArgsConstructor
7  @Node("CITY")
8  public class CityNode {
9      @Id
10     private String id;
11     private String name;
12     private String country;
13 }

```

Listing 7.26: City Node Implementation

Hotel Node

```

1  /**
2   * Neo4j Node entity representing a Hotel accommodation.
3   * Utilizes an outgoing LOCATED_AT relationship to a CityNode.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  @Node("Hotel")
9  public class HotelNode {

```

```

10     @Id
11     private String id;
12     private String name;
13     private String city;
14     private GeographicPoint2d location;
15
16     @Relationship(type = "LOCATED_AT", direction = Relationship.Direction.
17     OUTGOING)
18     private CityNode cityNode;
19 }

```

Listing 7.27: Hotel Node Implementation

POI Node

```

1  /**
2   * Neo4j Node entity representing a specific Point of Interest.
3   * Defines relationships such as BELONGS_TO and LOCATED_IN.
4   */
5  @Data
6  @AllArgsConstructor
7  @NoArgsConstructor
8  @Node("POI")
9  public class POINode {
10     @Id
11     private String id;
12     private String name;
13     private String category;
14     private String city;
15     private GeographicPoint2d location;
16
17     @Relationship(type = "BELONGS_TO", direction = Relationship.Direction.
18     OUTGOING)
19     private CategoryNode categoryNode;
20     @Relationship(type = "LOCATED_IN", direction = Relationship.Direction.
21     OUTGOING)
22     private CityNode cityNode;
23 }

```

Listing 7.28: POI Node Implementation

Travel Package Node

```

1  /**
2   * Neo4j Node entity representing a Travel Package.
3   * There are outgoing relationships to Categories, POIs, Destinations, and
4   * Hotels.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  @Node("TravelPackage")
10 public class TravelPackageNode {
11     @Id
12     private String id;
13     private String title;
14     private Double price;
15     private Double avgRating;
16
17     @Relationship(type = "HAS_CATEGORY", direction = Relationship.Direction.
18     OUTGOING)
19 }

```

```

17     private List<CategoryNode> categories;
18     @Relationship(type = "INCLUDES", direction = Relationship.Direction.
19     OUTGOING)
20     private List<POINode> pois;
21     @Relationship(type = "DESTINATION", direction = Relationship.Direction.
22     OUTGOING)
23     private CityNode destination;
24     @Relationship(type = "HAS_HOTEL", direction = Relationship.Direction.
25     OUTGOING)
26     private HotelNode hotel;
27 }

```

Listing 7.29: Travel Package Node Implementation

User Node

```

1  /**
2   * Neo4j Node entity representing a User in the graph.
3   * Maintains stateful relationships with properties (@RelationshipProperties)
4   * pointing to the
5   * packages they have PURCHASED and REVIEWED.
6   */
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 @Node("User")
11 public class UserNode {
12     @Id
13     private String id;
14     @Property("first_name")
15     private String firstName;
16     @Property("last_name")
17     private String lastName;
18     @Property("birth_date")
19     private LocalDate birthDate;
20     private String role;
21
22     @Relationship(type = "REVIEWED", direction = Relationship.Direction.
23     OUTGOING)
24     private List<ReviewedRelationship> reviewedPackages;
25     @Relationship(type = "PURCHASED", direction = Relationship.Direction.
26     OUTGOING)
27     private List<PurchasedRelationship> purchasedPackages;
28 }

```

Listing 7.30: User Node Implementation

Purchased Relationship

```

1  /**
2   * Neo4j Relationship entity representing the "PURCHASED" action.
3   * Connects a UserNode to a TravelPackageNode and stores the temporal
4   * property (timestamp) of the transaction.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  @RelationshipProperties
10 public class PurchasedRelationship {
11     @RelationshipId
12     private String internalId;
13 }

```

```

12
13     @TargetNode
14     private TravelPackageNode travelPackage;
15     private LocalDateTime timestamp;
16 }

```

Listing 7.31: Purchased Relationship Implementation

Reviewed Relationship

```

1  /**
2   * Neo4j Relationship entity representing the "REVIEWED" action.
3   * Connects a UserNode to a TravelPackageNode, storing the assigned rating
4   * and the timestamp.
5   */
6  @Data
7  @AllArgsConstructor
8  @NoArgsConstructor
9  @RelationshipProperties
10 public class ReviewedRelationship {
11     @RelationshipId
12     private String internalId;
13
14     @TargetNode
15     private TravelPackageNode travelPackage;
16     private Integer rating;
17     private LocalDateTime timestamp;
18 }

```

Listing 7.32: Reviewed Relationship Implementation

7.5 Repository

The repository layer is responsible for interacting with the databases, specifically MongoDB and Neo4j. To maximize efficiency and flexibility, standard queries utilize Spring Data's declarative method derivation, whereas for custom queries in MongoDB and Neo4j, the `@Query` and `@Aggregation` annotations are used to handle advanced operations.

7.5.1 MongoDB Repositories

These interfaces extend `MongoRepository` and handle standard operations alongside highly complex aggregation pipelines:

User_MongoInterface

Manages user records and authentication checks. It includes complex `@Aggregation` pipelines designed for administrative analytics, such as computing the total monthly revenue from confirmed orders and identifying the top platform customers based on purchase volume.

TravelPackage_MongoInterface & Custom Implementation

Responsible for fetching travel packages. It implements a custom Java-based aggregation logic (placed in `TravelPackage_Custom_MongoInterfaceImpl`) that constitutes the core scoring algorithm for the application. This logic filters packages based on user constraints (budget, reachable cities, season) and computes a match score prioritizing interests, ratings, and price. It also features a native aggregation pipeline designed for administrative analytics, to discover shared structural elements (e.g., hotels) among highly-rated packages.

Flight_MongoInterface

Handles spatio-temporal queries over the flights collection. It exposes methods to pre-fetch reachable destinations and retrieve valid outbound and return flights based on strict time boundaries.

Questionnaire_MongoInterface

It incorporates an `@Aggregation` pipeline that aggregates questionnaires by the customer's age demographic range, calculates the average budget, tracks totals financial outputs before executing sorting pipelines.

Review_MongoInterface

It defines an `@Aggregation` pipeline designed to calculate real-time success metrics for surprise packages. It matches surprise documents, groups results to sum total 4-star and 5-star reviews, and outputs system success rates to evaluate the personalization algorithm's performance.

7.5.2 Neo4j Graph Repositories

These interfaces extend `Neo4jRepository` and coordinate native Cypher queries to navigate the graph:

Recommendation_Neo4jInterface

The core recommendation algorithm interface. It executes sophisticated Cypher queries for interest-based filtering (recommending packages based on shared categories with previously loved trips), age-based filtering (suggesting packages highly rated by other users in the same age range), Popularity trends, and Geospatial recommendations (calculating distances between the booked hotel and nearby POIs).

Admin_Analytics_Neo4jInterface

It executes a sophisticated Cypher query, exploiting complex node traversals to identify the most popular destinations categorized by specific user age groups.

Sync_Neo4jInterface

The synchronization central part. It exposes `MERGE` and `DELETE` operations to create or remove users, and registers `PURCHASED` and `REVIEWED` relationships, keeping the graph state perfectly aligned with MongoDB.

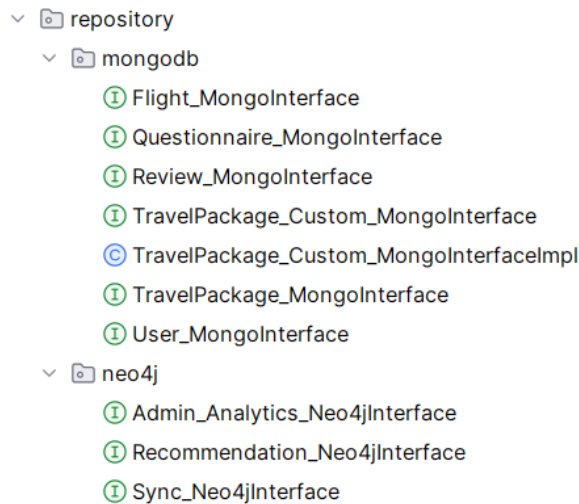


Figure 7.3: Repository package folder.

7.6 Service

The service layer serves as the logical operational intelligence of the application. It acts as a crucial intermediary between the controllers and the repositories.

Furthermore, it manages transactional operations and enforces a strict separation of concerns.

7.6.1 MongoDB Services

User_Service_Mongo

It coordinates client registering, authentication, and security updates. Upon user registration, it evaluates personal records, verifies email and name uniqueness against the database, and utilizes the Google Guava library to securely compute a cryptographic SHA-256 hash of the plain-text password before document persistence. Importantly, it initiates data synchronization by compiling a `SYNC_NEW_USER` task payload and pushing it into the `Neo4jSyncManager`. It also handles updating profiles and account deletions, safely pushing a deletion event (`DELETE_USER_NEO4J`) to wipe the user's graph footprint.

CartAndOrder_Service_Mongo

It manages operational cart interaction and order transactions. It calculates package totals by appending selected flight costs, handling proper precision mapping. This service is the owner of the "Surprise Trip Protection" mechanism for the frontend. During the execution of read operations (`getUserCart` and `getUserHistory`), the service intercepts the returning entities. If an item is flagged as a surprise trip and the current calendar date has not yet surpassed the flight's departure date, the service overrides properties in RAM; in addition, it substitutes the title with a censored title, masks real cities with "Secret Destination", and overwrites flight codes with question marks. This strategy completely isolates data masking to the runtime memory level, ensuring that data stored in MongoDB remains integer and mathematically accurate for analytics, while the client response remains perfectly secure against data leaks.

Questionnaire_Service_Mongo

It handles the questionnaire logic and calculates the travel package suggestions based on user preferences and live flight availability. When a questionnaire is compiled, it releases a pre-fetch call to find reachable destinations, runs the custom scoring algorithm method to capture

the top 3 best matching packages, and calls internal private method (`enrichWithLiveFlights`) to automatically pair real-time flight availability with the matching package snapshots.

TravelPackage_Service_Mongo

A lightweight service designed to retrieve the full details of available travel packages and load the specific subset of packages suggested during a user's previous questionnaire session.

Review_Service_Mongo

It intercepts customer review submissions. Upon saving a new review record, it triggers an internal synchronization method (`syncPackageReviewsAndRating`) that loads the corresponding package document, updates a restricted embedded list containing only the latest 3 reviews to optimize landing page read speeds, recalculates the running package average rating, and enqueues a graph synchronization request (`ADD_REVIEWED_RELATION`).

Admin_Analytics_Service_Mongo

It coordinates analytics reporting for administrative management. It constructs a multi-stage pipeline to discover negative user feedback, it also computes revenue, and it simulates email marketing automation by executing sorting pipelines over customer purchase weights. Later on, it automatically generates personalized promotional coupon codes, and printing formatted transmission logs to the system console.

7.6.2 Neo4j Graph Services & Distributed Synchronization

- **Recommendation_Service_Neo4j**: This is a light transactional proxy layer that not only safeguards the input parameters but also sends them to the graph repositories. In simple terms, it directs specific recommendation listings to various client endpoints, such as the popularity and the for you page.
- **Admin_Analytics_Service_Neo4j**: it provides an abstraction layer for administrative graph analysis, returning demographic age-driven marketing reports.

Neo4jSyncManager

This service acts as a strategic optimization component designed to execute resource-intensive, eventual-consistency operations between MongoDB and Neo4j, without degrading main application performance. It relies on a thread-safe concurrent queue `ConcurrentLinkedQueue` to store deferred background tasks, periodically evaluating the underlying operating system's CPU load to determine execution safety.

To guarantee that intensive graph mutations don't degrade real-time user traffic or exhaust server resources, the class utilizes Spring's task scheduling library to execute a background thread every 10 seconds. Upon invocation, the manager interfaces with Java's native `ManagementFactory` API to inspect the host system's real-time CPU load via a specific interface called `OperatingSystemMXBean`.

If the current CPU load is safely below a strict threshold (40%), it authorizes data processing, polling and executing a batch of up to 10 tasks sequentially, by mapping them to specialized native repository commands. If the CPU load exceeds the safety boundary, the manager defers execution to the next cycle, effectively implementing an adaptive and resource-aware mechanism.

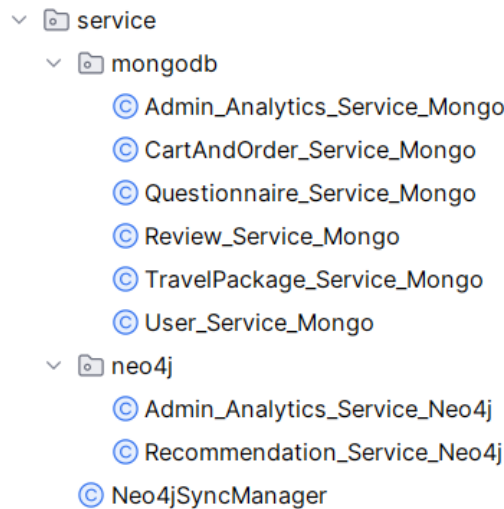


Figure 7.4: Service package folder.

7.7 Controller

The controller layer forms the external boundary of the application, exposing standardized RESTful API endpoints to handle incoming client-side HTTP traffic. These classes handle basic data validation, parse incoming request bodies into transfer objects, map Path Variables and Request Parameters, and wrap service returns within appropriate Spring `ResponseEntity` structures, to ensure explicit HTTP status code routing.

MongoDB Controllers

User_Controller_Mongo

It exposes public POST endpoint for user registration (`/register`), authentication route (`/login`) validating user states, GET endpoint (`/userId`) to view profiles, PUT path (`/userId/settings`) for updating sensitive credentials or documents, and DELETE method (`/userId`) to request permanent account erasure.

CartAndOrder_Controller_Mongo

It hosts the endpoints for the user purchasing activities. It provides POST path (`/cart/items`) to add specific travel packages and flights to the shopping cart, GET endpoints (`/cart`) to review the current cart and (`/orders`) to view the order's history, a POST endpoints (`/cart/checkout`) to execute transaction processes and (`/orders/{orderId}/add-poi`) to add specific pois to the current order, a DELETE endpoint (`/remove/{packageId}`) to remove a specific travel package from the user's shopping cart.

It also exposes the GET path `/orders/{orderId}/flights`, which delivers a unified flight summary details.

Questionnaire_Controller_Mongo

It manages questionnaire interactions, exposing POST path (`/user/submit`) to submit preference sheets, PUT endpoint (`/id/select-package`) that allow users to bind their selection to a specific suggested package ID on the recent questionnaire and GET endpoint (`/user/{userId}`) that retrieves all questionnaires associated with a specific user.

Review_Controller_Mongo

It provides POST endpoint for users to log package ratings and reviews, alongside a GET path (`/package/{packageId}`) that retrieves all approved reviews associated with a specific travel package.

TravelPackage_Controller_Mongo

It provides GET endpoint (`/packageId`) to explore individual travel package details by ID, alongside a GET path (`/suggested/{userId}`) designed to load the target package cards generated during the user's latest questionnaire session.

Admin_Analytics_Controller_Mongo

It exposes GET endpoints restricted to the admin to extract negative review reports, evaluate surprise package performance logs, retrieve monthly revenues, and trigger simulated customer loyalty marketing reward campaigns.

Neo4j Graph Controllers

Recommendation_Controller_Neo4j

It acts as the gateway to the graph recommendation subsystem. It maps dedicated GET endpoints that provide the front-end user interface with algorithmic personalized feeds, including globally trending popular travel packages, interest-targeted travel packages, age demographic suggestions, and geospasial point-of-interest arrays localized within a 5km radius of a booked hotel.

Admin_Analytics_Controller_Neo4j

It exposes the graph analytics dashboard endpoint, enabling the administrator to view destination popularity statistics grouped by demographic age ranges via native Neo4j traversals.

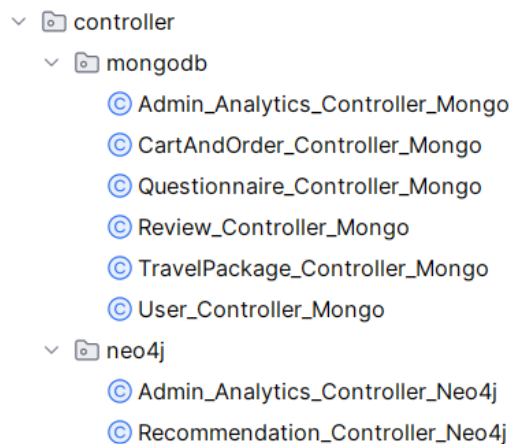


Figure 7.5: Controller package folder.

7.8 Utils

The `utils` package encapsulates standalone and stateless utility modules that decouple specific algorithmic formatting logic from core business service files.

SeasonUtils

It provides a centralized framework for translating temporal constraints. It exposes static methods that accept text-based season strings submitted by users, and maps them to hard-coded Java **Instant** object arrays.

These arrays specify the exact ISO UTC date boundaries (upper and lower bounds) relative to the upcoming calendar year, providing the precise parameters required by the MongoDB flight engine to isolate valid flight schedules.



Figure 7.6: Utils package folder.

Chapter 8

Relevant Queries

In this chapter, only the most relevant queries, particularly those involving complex business logic or significant computational load, will be discussed. For this reason, classic CRUD (Create, Read, Update, Delete) operations are not analyzed in detail.

8.1 MongoDB Queries

8.1.1 Retrieve Negative Reviews per Package

This query extracts and analyzes negative reviews for each package available on the platform. To accurately identify negative feedback, the query applies a dual-condition filter: it flags a review as negative only if it has a low rating (3 or less) and explicitly contains at least one negative keyword within the text comment. This semantic filtering approach helps to exclude mixed or neutrally phrased low-rating reviews, providing a more reliable metric of user dissatisfaction. Ultimately, platform administrator can leverage these clear insights to identify critical pain points, address specific issues, and proactively improve the overall customer experience.

Each entry in the result contains the following information:

- **_id**: the unique identifier of the package (`package_id`)
- **packageName**: the title of the package
- **total_reviews**: the overall number of reviews received by the package
- **total_negative_found**: the total count of reviews explicitly flagged as negative
- **reviews**: an array containing the detailed information of the negative reviews (including user, rating, and text comment)
- **negative_percentage**: the percentage of negative reviews relative to the total

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **AddFields - Negative Flagging**: The first stage evaluates every review using the `$addFields` operator to create a boolean `is_negative` flag. It uses the `$and` operator to enforce two strict conditions: the `$rating` must be less than or equal to 3, and the `$comment` must match a predefined regular expression of negative keywords (e.g., *dirty*, *bad*, *terrible*, *worst*) using `$regexMatch` with the case-insensitive option (`i`).

2. **Group - Package Aggregation:** This stage groups the documents by `package_id`. It retains the package title using `$first` and computes global statistics, such as the total number of reviews (`$sum: 1`). It also counts the negative reviews by conditionally summing 1 if `is_negative` is true. Simultaneously, it uses the `$push` operator combined with `$cond` to append only the negative reviews into a dedicated array, dynamically discarding the positive or neutral ones by utilizing the `$$REMOVE` system variable.
3. **Project – Percentage Calculation and Formatting:** The final stage shapes the output document. It retains the previously computed fields and introduces a new `negative_percentage` field.

```

1 AggregationOperation addFieldsStage = context -> Document.parse(
2     "{ $addFields: { " +
3         "         is_negative: { " +
4             "             $and: [ " +
5                 "                 { $lte: ['$rating', 3] }, " +
6                 "                 { $regexMatch: { input: '$comment', regex: 'dirty|bad|delay|
worst|terrible|disappointment|poor|dissatisfied|waste|average', options: '
i' } } " +
7             "         ] " +
8         "     } " +
9     "}" );
10 );
11
12 AggregationOperation groupStage = context -> Document.parse(
13     "{ $group: { " +
14         "         _id: '$package_id', " +
15         "         packageName: { $first: '$package_title' }, " +
16         "         total_reviews: { $sum: 1 }, " +
17         "         total_negative_found: { $sum: { $cond: ['$is_negative', 1, 0] } }, "
18     +
19     "         reviews: { " +
20         "             $push: { " +
21             "                 $cond: [ " +
22                 "                     '$is_negative', " +
23                 "                     { user: '$user_id', rating: '$rating', comment: '$comment' },
24                 "                 ] " +
25             "             } " +
26         "         } " +
27     "}" );
28 );
29
30 AggregationOperation projectStage = context -> Document.parse(
31     "{ $project: { " +
32         "         _id: 1, " +
33         "         packageName: 1, " +
34         "         total_reviews: 1, " +
35         "         total_negative_found: 1, " +
36         "         reviews: 1, " +
37         "         negative_percentage: { " +
38         "             $round: [ " +
39         "                 { $multiply: [ { $divide: ['$total_negative_found', '
$total_reviews' ] }, 100 ] }, " +
40         "                 2 " +
41             "         ] " +
42         "     } " +
43     "}" );
44 );

```

Listing 8.1: Aggregation Pipeline for Negative Reviews Analysis

8.1.2 Surprise Package Success Rate

This query evaluates the true effectiveness of the platform's personalization algorithm by analyzing user feedback specifically on "surprise" travel packages. "Surprise" packages are purchased entirely blind. Therefore, high satisfaction rates for these blind purchases serve as the most accurate metric of the algorithm's matching efficiency, proving that the system successfully interpreted the user's preferences without relying on their manual selection. Each entry in the result contains the following information:

- **total_reviews**: the overall number of reviews left for surprise packages
- **five_star_count**: the exact number of surprise packages that received a perfect 5-star rating
- **four_star_count**: the exact number of surprise packages that received a 4-star rating
- **total_success_rate**: the percentage of overall positive reviews (4 or 5 stars) relative to the total surprise packages
- **excellence_rate**: the percentage of strictly 5-star reviews, representing the algorithm's highest accuracy

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match -Surprise Packages Filtering**: The first stage filters the entire dataset using the `$match` operator to include only the reviews associated with a surprise package (`'is_surprise': true`).
2. **Group – Global Aggregation and Counting**: This stage groups all incoming documents into a single global result by setting `_id` to `null`. It calculates the total number of reviews left for surprise packages (`$sum: 1`). Simultaneously, it uses the `$cond` operator within a `$sum` accumulator to count specific rating tiers: it counts 5-star reviews, 4-star reviews, and the total positive reviews (defined as any rating greater than or equal to 4).
3. **Project - Metric Calculation and Renaming**: The final stage shapes the output document. It suppresses the `_id` field (set to 0) and renames the aggregated counts for better readability (e.g., mapping `total_surprises` to `total_reviews`). Crucially, it calculates two Key Performance Indicators (KPIs) by evaluating the following equations, rounding both to two decimal places using the `$round` operator:

$$Total_Success_Rate = \text{round} \left(\left(\frac{\text{total_positive}}{\text{total_reviews}} \right) \cdot 100, 2 \right)$$

$$Excellence_Rate = \text{round} \left(\left(\frac{\text{five_stars}}{\text{total_reviews}} \right) \cdot 100, 2 \right)$$

```

1 @Aggregation(pipeline = {
2   "{ $match: { 'is_surprise': true } }",
3   "{ $group: { " +
4     "_id: null, " +
5     "total_surprises: { $sum: 1 }, " +
6     "five_stars: { $sum: { $cond: [{ $eq: ['$rating', 5] }, 1, 0] } }, "
+

```

```

7      "      four_stars: { $sum: { $cond: [{ $eq: ['$rating', 4] }, 1, 0] } }, "
8      +
9      "      total_positive: { $sum: { $cond: [{ $gte: ['$rating', 4] }, 1, 0] }
10     } " +
11     "    } }",
12     "{ $project: { " +
13     "      _id: 0, " +
14     "      total_reviews: '$total_surprises', " +
15     "      five_star_count: '$five_stars', " +
16     "      four_star_count: '$four_stars', " +
17     "      total_success_rate: { $round: [ { $multiply: [ { $divide: ['
18     $total_positive', '$total_surprises'] }, 100 ] }, 2 ] }, " +
19     "      excellence_rate: { $round: [ { $multiply: [ { $divide: ['$five_stars
20     ', '$total_surprises'] }, 100 ] }, 2 ] } " +
21     "    } }"
22   })
23   SurprisePackageStatsDTO getSurprisePackageStats();

```

Listing 8.2: Aggregation Pipeline for Surprise Package Stats

8.1.3 Average Budget by Age Range

This query analyzes user spending power across different demographic groups based on their submitted questionnaires. By grouping the data according to age ranges, the platform can determine the average budget limit for each demographic segment. This insight is highly valuable for marketing and product strategies, allowing administrator to tailor travel offerings, optimize pricing tiers, and align future packages with the actual financial expectations of their target audiences.

Each entry in the result contains the following information:

- **_id**: the age range category (e.g., "18-24", "25-34") extracted from the questionnaire
- **total_questionnaires**: the total number of users (questionnaires) falling into this specific age group
- **average_budget**: the calculated average budget limit for the demographic group, properly formatted

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Group – Demographic Aggregation**: This stage groups the questionnaires based on the `preferences.age_range` field. For each distinct age bracket, it calculates the raw average of the users' maximum spending capacities (`preferences.budget_limit`) using the `$avg` operator. Additionally, it computes the size of the demographic segment by counting the total number of questionnaires using the `$sum: 1` accumulator.
2. **Project – Data Formatting and Rounding**: This stage refines the output document. It explicitly retains the `_id` and `total_questionnaires` fields. Furthermore, it transforms the mathematically precise `raw_average` into a clean `average_budget` field by applying the `$round` operator, truncating the value to exactly two decimal places to represent standard currency formatting.

3. **Sort – Sequential Ordering:** The final stage applies the `$sort` operator to arrange the results in ascending order (1) based on the `_id` field (the age range). This ensures that the final dataset is presented logically, typically from the youngest demographic to the oldest, making it immediately readable for frontend charts and reports.

```

1 @Aggregation(pipeline = {
2   "{ $group: { _id: '$preferences.age_range', raw_average: { $avg: '
   $preferences.budget_limit' }, total_questionnaires: { $sum: 1 } } }",
3   "{ $project: { _id: 1, total_questionnaires: 1, average_budget: { $round:
   [ '$raw_average', 2 ] } } } }",
4   "{ $sort: { '_id': 1 } }"
5 })
6 List<BudgetByAgeRangeDTO> getAverageBudgetByAgeRange();

```

Listing 8.3: Aggregation Pipeline for Average Budget by Age Range

8.1.4 Monthly Revenue and Order Volume Analysis

This query calculates the platform's total monthly revenue and the overall volume of successful transactions. By aggregating the financial data of all confirmed orders, platform administrator can track economic performance over time, identify seasonal sales trends, and monitor business growth.

Each entry in the result contains the following information:

- **Year:** the year the orders were processed
- **Month:** the month the orders were processed
- **Total_Revenue:** the total revenue generated during that specific month, properly rounded to two decimal places
- **Number_of_orders:** the total count of confirmed orders completed within that month

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match -Pre-Filtering:** The pipeline begins by filtering the dataset using the `$match` operator to include only documents containing at least one order with an `order_status` of `confirmed`. This initial pass optimizes performance by discarding users or carts with no successful transactions before executing the resource-intensive unwind stage.
2. **Unwind -Array Deconstruction:** The `$unwind` operator deconstructs the `orders` array. This step transforms a single user document containing multiple orders into multiple distinct documents, each representing one individual order.
3. **Match -Exact Order Filtering:** A second `$match` stage is applied to strictly retain only the specific orders whose status is `confirmed`. Because the preceding unwind operation exposes every order (including pending or failed ones that belonged to the initially matched users).

4. **Group – Time-Based Aggregation:** The documents are grouped temporally to create monthly summaries. The `_id` is defined as a composite key that extracts the `$year` and `$month` from the `orders.purchase_date`. For each time group, it calculates the monthly revenue (`fatturato_mensile`) by summing the individual order prices (`$sum: '$orders.price'`) and counts the total number of processed orders using the `$sum: 1` accumulator.
5. **Project - Data Shaping and Formatting:** This stage formats the final output document for the application layer. It suppresses the complex `_id` object (`_id: 0`) and flattens the date structure into explicit `Anno` and `Mese` fields. Furthermore, it ensures the `Fatturato_Totale` is cleanly formatted as a standard currency value by rounding it to two decimal places using the `$round` operator.
6. **Sort - Chronological Ordering:** The `$sort` operator arranges the results in descending order (`-1`) by year and month.

```

1 @Aggregation(pipeline = {
2   "{ $match: { 'orders.order_status': 'confirmed' } }",
3   "{ $unwind: '$orders' }",
4   "{ $match: { 'orders.order_status': 'confirmed' } }",
5   "{ $group: { _id: { year: { $year: '$orders.purchase_date' }, month: {
6     $month: '$orders.purchase_date' } }, fatturato_mensile: { $sum: '$orders.
7     price' }, numero_ordini: { $sum: 1 } } }",
8   "{ $project: { _id: 0, Anno: '$_id.year', Mese: '$_id.month',
9     Fatturato_Totale: { $round: ['$fatturato_mensile', 2] }, Numero_Ordini: '
    $numero_ordini' } }",
10  "{ $sort: { Anno: -1, Mese: -1 } }"
11 })
12 List<MonthlyRevenueDTO> getMonthlyRevenue();

```

Listing 8.4: Aggregation Pipeline for Monthly Revenue and Order Volume Analysis

8.1.5 Top Customers Loyalty Campaign

This query identifies the platform's most loyal customers by calculating the exact number of successfully completed purchases for each user. In this way, the administrator can automatically reach out to the top 10 most active users on the platform and reward their loyalty with a 10% discount on their next trip. This proactive approach helps to foster brand loyalty and encourage repeat bookings.

Each entry in the result contains the following information:

- **_id:** the unique identifier of the user
- **first_name:** the customer's first name
- **last_name:** the customer's last name
- **email:** the customer's email address, which is essential for sending the promotional discount code
- **confirmed_orders:** the total count of successfully confirmed orders placed by the user

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match – Active Shoppers Filtering:** The pipeline begins by filtering the dataset using the `$match` operator to include only users who have an `orders` array that exists (`$exists: true`) and is not empty (`$ne: []`). This initial step optimizes performance by immediately discarding users who registered but never initiated a purchase.
2. **Project – Data Extraction and Array Filtering:** This stage shapes the output document and calculates the primary metric. It extracts the user's personal details (`first_name`, `last_name`) and contact information (`email`). To determine the exact number of successful purchases, it calculates the `confirmed_orders` field dynamically: first, it uses the `$filter` operator to iterate through the user's `orders` array, retaining only those specific elements where the `order_status` is exactly `confirmed`. Then, it applies the `$size` operator to count the remaining elements in this filtered array.
3. **Sort – Ranking Generation:** The `$sort` operator arranges the resulting documents in descending order (`-1`) based on the newly calculated `confirmed_orders` field. This ensures that the most frequent buyers are positioned at the very top of the list.
4. **Limit – Top 10 Selection:** The final stage uses the `$limit` operator to restrict the output to exactly 10 documents. This effectively truncates the list, isolating only the absolute top 10 most loyal customers eligible for the promotional campaign.

```

1 @Aggregation(pipeline = {
2   "{ $match: { 'orders': { $exists: true, $ne: [] } } }",
3   "{ $project: { _id: 1, first_name: '$personal_info.first_name', last_name
4     : '$personal_info.last_name', email: '$credentials.email',
5     confirmed_orders: { $size: { $filter: { input: '$orders', as: 'order',
6       cond: { $eq: ['$order.order_status', 'confirmed'] } } } } } }",
7   "{ $sort: { confirmed_orders: -1 } }",
8   "{ $limit: 10 }"
9 })
10 List<TopCustomerDTO> getTopCustomersForDiscount();

```

Listing 8.5: Aggregation Pipeline for Top Customers Loyalty Campaign

8.2 Neo4j Queries

8.2.1 Popular Packages Feed Generation

This query drives the platform's "Popular" section by dynamically identifying the most highly-rated travel packages based on recent user feedback. To ensure that the recommendations reflect current trends and maintain high quality, the algorithm restricts the analysis to reviews submitted within the last six months. Furthermore, it enforces a statistical significance threshold, requiring a package to have a minimum number of reviews to be considered. In this way, the administrator and the system can guarantee that the homepage showcases genuinely popular destinations, preventing newly added packages with a single 5-star rating from artificially dominating the rankings.

Each entry in the result contains the following information:

- **packageId:** the unique identifier of the travel package
- **packageName:** the title of the travel package
- **price:** the cost of the package
- **avgRating:** the calculated average rating based on the last six months of feedback

- **totalReviews**: the exact number of reviews received during the analyzed timeframe

Query Execution Description

The Cypher query performs the following steps:

1. **MATCH – Graph Traversal**: The query begins by traversing the graph database to find specific interaction patterns. It looks for **User** nodes connected to **TravelPackage** nodes via a **REVIEWED** relationship, effectively isolating the feedback data.
2. **WHERE – Temporal Filtering**: A temporal condition is applied directly to the relationship's **timestamp** property. It filters the interactions to include only reviews submitted within the last six months (`datetime() - duration({months: 6})`). This ensures the popularity metrics are based on current, relevant data rather than historical lifetime scores.
3. **WITH – Data Aggregation**: The **WITH** clause acts as an intermediate aggregation step. It groups the matched patterns by the package node (**p**) and computes two crucial metrics: it counts the total number of valid reviews (`count(r)`) and calculates the average rating (`avg(r.rating)`), cleanly rounding the result to two decimal places.
4. **WHERE – Statistical Threshold**: A second **WHERE** clause filters the newly aggregated data, retaining only the packages that have accumulated at least 10 reviews (`totalReviews >= 10`). This acts as a quality control mechanism to ensure the average score is statistically significant.
5. **RETURN – Data Extraction**: This stage shapes the final output document. It extracts the package's ID, title, and price from the node properties, and pairs them with the computed `avgRating` and `totalReviews`.
6. **ORDER BY and LIMIT – Ranking Generation**: Finally, the query ranks the packages. It sorts the results in descending order primarily by the highest average rating, using the total number of reviews as a secondary tie-breaker. The **LIMIT 15** clause restricts the output to the top 15 packages, making the dataset perfectly sized for a frontend carousel or list.

```

1 @Query("MATCH (u:User)-[r:REVIEWED]->(p:TravelPackage) " +
2       "WHERE r.timestamp >= datetime() - duration({months: 6}) " +
3       "WITH p, round(avg(r.rating), 2) AS avgRating, count(r) AS
4       totalReviews " +
5       "WHERE totalReviews >= 10 " +
6       "RETURN p.id AS packageId, p.title AS packageName, p.price AS price,
7       avgRating, totalReviews " +
8       "ORDER BY avgRating DESC, totalReviews DESC LIMIT 15")
9 List<PopularPackageDTO> getPopularPackages();

```

Listing 8.6: Cypher Query for Popular Packages Analysis

8.2.2 Demographic Filtering Recommendations

This query generates personalized travel package recommendations for the "For You" section by implementing a demographic-based filtering algorithm. The system calculates the current age of the target user to assign them to a specific demographic cohort (e.g., Under 30, between 30-50, Over 50) and analyzes the purchasing history of other users within the same bracket. The query identifies packages with an average rating of 4 stars or higher that have

been purchased by these demographic peers, while explicitly excluding any packages the target user has already bought. To ensure statistical reliability in the final ranking, a Bayesian weighted score is applied to balance the raw average rating against the total volume of reviews.

Each entry in the result contains the following information:

- **packageId**: the unique identifier of the recommended travel package
- **suggestedPackage**: the title of the travel package
- **price**: the cost of the package
- **avgRating**: the raw average rating of the package globally
- **numberOfReviews**: the total number of reviews the package has received platform-wide
- **countSimilarAgeTarget**: the exact number of users in the same demographic cohort who purchased this package
- **analyzedAgeTarget**: the target age cohort applied during the query execution (e.g., "Under 30")

Query Execution Description

The Cypher query performs the following steps:

1. **MATCH – Target Age Calculation**: The query begins by locating the specific target user (`$userId`). It dynamically calculates their current age using the `duration.between` function comparing their `birth_date` to the current date.
2. **WITH – Demographic Bucket Generation**: Through a series of `CASE` statements, the query categorizes the target user into a logical age cohort and simultaneously computes the upper and lower date boundaries (`date_bounds`) required to search for users born in the same era.
3. **MATCH – Peer Identification**: The algorithm locates other users (`others`) whose birth dates fall strictly within the generated `date_bounds`, explicitly ensuring the original target user is excluded from this group (`others.id <> target.id`).
4. **MATCH – Collaborative Filtering**: The query traverses the graph to find `TravelPackage` nodes that the identified peers have purchased. A strict `WHERE NOT` pattern ensures that packages already bought by the target user are discarded. Furthermore, it filters for high quality by requiring an `avg_rating` of at least 4.
5. **WITH – Aggregation and Subqueries**: The query groups the results to count how many peers bought each specific package (`countSimilarAgeTarget`). Simultaneously, it utilizes an inline subquery (`count { ()-[:REVIEWED]->(p) }`) to efficiently retrieve the total number of global reviews for the package without altering the cardinality of the main execution plan.
6. **WITH – Bayesian Score Calculation**: To prevent highly-rated but rarely-reviewed packages from surpassing established ones, a weighted score is computed dynamically using the following Bayesian approximation:

$$WeightedScore = \frac{AvgRating \cdot NumberOfReviews + 20.0}{NumberOfReviews + 5.0}$$

7. **RETURN and ORDER BY – Result Formatting:** The final stage shapes the result set. It orders the recommendations primarily by the volume of peer purchases in descending order, using the computed `weightedScore` as a secondary tie-breaker. The `LIMIT 10` clause ensures the frontend feed receives a manageable list of the absolute best matches.

```

1 @Query("MATCH (target:User {id: $userId}) " +
2   "WITH target, duration.between(target.birth_date, date()).years AS
   target_age " +
3   "WITH target, target_age, " +
4   "   CASE " +
5   "     WHEN target_age < 30 THEN 'Under 30' " +
6   "     WHEN target_age >= 30 AND target_age <= 50 THEN 'between 30-50' "
   +
7   "     ELSE 'Over 50' " +
8   "   END AS target_age_group, " +
9   "   CASE " +
10  "     WHEN target_age < 30 THEN [date() - duration({years: 30}), date()
   ] " +
11  "     WHEN target_age >= 30 AND target_age <= 50 THEN [date() -
   duration({years: 50}), date() - duration({years: 30})] " +
12  "     ELSE [date() - duration({years: 120}), date() - duration({years:
   50})] " +
13  "   END AS date_bounds " +
14  "MATCH (others:User) " +
15  "WHERE others.id <> target.id " +
16  "   AND others.birth_date > date_bounds[0] " +
17  "   AND others.birth_date <= date_bounds[1] " +
18  "MATCH (others)-[:PURCHASED]->(p:TravelPackage) " +
19  "WHERE NOT (target)-[:PURCHASED]->(p) AND p.avg_rating >= 4 " +
20  "WITH p, target_age_group, count(others) AS countSimilarAgeTarget " +
21  "WITH p, target_age_group, countSimilarAgeTarget, count { ()-[:
   REVIEWED]->(p) } AS numberOfReviews " +
22  "WITH p, target_age_group, countSimilarAgeTarget, numberOfReviews, " +
23  "   (p.avg_rating * numberOfReviews + 20.0) / (numberOfReviews +
   5.0) AS weightedScore " +
24  "RETURN p.id AS packageId, " +
25  "   p.title AS suggestedPackage, " +
26  "   p.price AS price, " +
27  "   p.avg_rating AS avgRating, " +
28  "   numberOfReviews, " +
29  "   countSimilarAgeTarget, " +
30  "   target_age_group AS analyzedAgeTarget " +
31  "ORDER BY countSimilarAgeTarget DESC, weightedScore DESC LIMIT 10")
32 List<AgeRecommendationDTO> getRecommendationsByAge(String userId);

```

Listing 8.7: Cypher Query for Demographic Recommendations

8.2.3 Interest-Based Filtering Recommendations

This query provides a recommendation engine for the "For You" section, utilizing an interest-based filtering approach. The algorithm identifies the user's most recently reviewed travel package, treating it as a proxy for their current travel interests. It then traverses the graph to find other packages that share the same categories as this "seed" package. To maintain relevance and quality, the query explicitly excludes any packages the user has already purchased, filters out the seed package itself, and strictly requires a minimum average rating of 4.0. The final ranking is stabilized using a rounded Bayesian weighted score.

Each entry in the result contains the following information:

- **packageId**: the unique identifier of the recommended travel package
- **recommendedPackage**: the title of the recommended package
- **avgRating**: the raw average rating of the package globally
- **numberOfReviews**: the total number of reviews the package has received platform-wide
- **sharedCategory**: the numerical count of categories overlapping between the seed package and the recommended package
- **sharedCategoryNames**: a list containing the specific names of the shared categories (useful for frontend explainability)
- **basedOnLastReview**: the title of the target user’s most recently reviewed package, serving as the baseline for the recommendation

Query Execution Description

The Cypher query performs the following steps:

1. **MATCH and WITH – Seed Identification**: The query begins by locating the target user (`$userId`) and traversing the `REVIEWED` relationships to all their evaluated packages. It orders these relationships by their `timestamp` in descending order and applies a `LIMIT 1` clause to isolate only the single most recently reviewed package (`last_reviewed`), establishing it as the seed for the algorithm.
2. **MATCH – Category Extraction**: The algorithm traverses the graph from the `last_reviewed` package through the `HAS_CATEGORY` relationship to identify all associated `Category` nodes.
3. **MATCH – Interest-Based Filtering**: From the identified `Category` nodes, the query traverses backward through the `HAS_CATEGORY` relationships to discover other `TravelPackage` nodes (`rec`) that share at least one category with the seed package.
4. **WHERE – Exclusion and Quality Threshold**: An anti-pattern and logical filter is applied to ensure data validity. It excludes the seed package itself (`rec <> last_reviewed`) and any package the user has already bought (`NOT (u)-[:PURCHASED]->(rec)`). It also enforces a quality baseline by requiring `rec.avg_rating >= 4`.
5. **WITH – Overlap Aggregation**: The query groups the results to compute the similarity metrics. It counts the number of distinct overlapping categories (`sharedCategory`) and collects their actual names into a list (`sharedCategoryNames`).
6. **WITH – Review Counting**: An inline subquery (`count { ()-[:REVIEWED]->(rec) }`) is executed to retrieve the total number of global reviews for the recommended package, keeping the main query’s cardinality intact.
7. **WITH – Bayesian Score Calculation**: Similar to the demographic engine, a Bayesian weighted score is calculated to balance the raw rating against the review volume, and then cleanly rounded to two decimal places:

$$WeightedScore = round\left(\frac{rec.avg_rating \cdot numberOfReviews + 20.0}{numberOfReviews + 5.0}, 2\right)$$

8. **RETURN and ORDER BY – Result Formatting:** The final stage projects the necessary fields for the application layer. The results are ordered primarily by the `weightedScore` in descending order. In case of a tie, the raw `avgRating` is used as a secondary sort parameter, followed by the `sharedCategory` count. The output is constrained to the top 10 recommendations.

```

1 @Query("MATCH (u:User {id: $userId})-[r:REVIEWED]->(last_reviewed:
2   TravelPackage) " +
3   "WITH u, last_reviewed, r ORDER BY r.timestamp DESC LIMIT 1 " +
4   "MATCH (last_reviewed)-[:HAS_CATEGORY]->(c:Category) " +
5   "MATCH (c)-[:HAS_CATEGORY]-(rec:TravelPackage) " +
6   "WHERE rec <> last_reviewed AND NOT (u)-[:PURCHASED]->(rec) AND rec.
7   avg_rating >= 4 " +
8   "WITH rec, last_reviewed, count(DISTINCT c) AS sharedCategory, collect
9   (DISTINCT c.name) AS sharedCategoryNames " +
10  "WITH rec, last_reviewed, sharedCategory, sharedCategoryNames, count {
11    ()-[:REVIEWED]->(rec) } AS numberOfReviews " +
12  "WITH rec, last_reviewed, sharedCategory, sharedCategoryNames,
13  numberOfReviews, " +
14  "    round((rec.avg_rating * numberOfReviews + 20.0) / (
15    numberOfReviews + 5.0), 2) AS weightedScore " +
16  "RETURN rec.id AS packageId, " +
17  "    rec.title AS recommendedPackage, " +
18  "    rec.avg_rating AS avgRating, " +
19  "    numberOfReviews, " +
20  "    sharedCategory, " +
21  "    sharedCategoryNames, " +
22  "    last_reviewed.title AS basedOnLastReview " +
23  "ORDER BY weightedScore DESC, avgRating DESC, sharedCategory DESC LIMIT
24  10")
25 List<RecommendedPackageDTO> getRecommendationsByInterests(String userId);

```

Listing 8.8: Cypher Query for Interest-Based Content Recommendations

8.2.4 Top Destinations by Age Range

This query performs an advanced demographic analysis to identify macro-trends in travel preferences across different age groups. The algorithm calculates the users' ages at the time of query execution, groups them into specific demographic cohorts, and evaluates the purchasing volume for each destination. To ensure the results reflect high-quality trends, the analysis strictly includes packages with a baseline rating of 4.0 or higher and applies a Bayesian weighted score to stabilize the rankings. Finally, the query formats the output as a ready-to-read leaderboard, extracting the top three destinations for each age group.

Each entry in the result contains the following information:

- **ageRange:** the designated demographic cohort (e.g., "1. Under 30", "2. Between 30-50", "3. Over 50"), formatted with numeric prefixes to ensure correct alphabetical sorting
- **firstPlace:** a formatted string detailing the most popular city in that cohort, including its total purchase count and rounded Bayesian score
- **secondPlace:** a formatted string detailing the second most popular city, or a placeholder ("-") if insufficient data exists
- **thirdPlace:** a formatted string detailing the third most popular city, or a placeholder ("-") if insufficient data exists

Query Execution Description

The Cypher query performs the following steps:

1. **MATCH and WHERE – Graph Traversal and Pre-Filtering:** The query traverses the graph from `User` nodes, through their `PURCHASED` `TravelPackages`, down to the destination `CITY`. It strictly filters out users without a recorded birth date and excludes low-rated packages (`avg_rating >= 4`).
2. **WITH – Age Calculation and Review Counting:** The algorithm calculates the exact age of each user using `duration.between`. Concurrently, an inline subquery retrieves the total number of reviews for the associated package (`pkg_reviews`).
3. **WITH – Demographic Cohort Assignment:** A `CASE` statement evaluates the calculated age and categorizes the user into one of three distinct cohorts.
4. **WITH – Destination Aggregation:** The dataset is grouped by the demographic cohort (`age_group`) and the city name (`destination`). For each grouping, it aggregates the total number of purchases (`count(pkg)`), the raw average rating (`avg(pkg.avg_rating)`), and the total volume of reviews (`sum(pkg_reviews)`).
5. **WITH – Bayesian Score Calculation:** To prevent destinations with a single, highly-rated package from skewing the results, the query computes a Bayesian weighted rating:

$$WeightedRating = \frac{raw_rating \cdot total_reviews + 20.0}{total_reviews + 5.0}$$

6. **ORDER BY and COLLECT – Leaderboard Generation:** The aggregated data is sorted by age group, then descending by purchase volume, and finally by the Bayesian score. The `collect(...)[0..3]` function is then used to aggregate the sorted destinations into a list and slice only the top three results for each age bracket.
7. **RETURN – Data Projection and String Formatting:** The final stage projects the output fields. It concatenates the city name, purchase count, and rounded score into human-readable strings. The `coalesce` function is employed for the second and third places to handle potential null values (e.g., returning "-" if a cohort only has one or two purchased destinations).

```

1 @Query("MATCH (u:User)-[:PURCHASED]->(pkg:TravelPackage)-[:DESTINATION]->(
2   city:CITY) " +
3   "WHERE u.birth_date IS NOT NULL AND pkg.avg_rating >= 4 " +
4   "WITH city, pkg, duration.between(date(u.birth_date), date()).years AS
   age, count { ()-[:REVIEWED]->(pkg) } AS pkg_reviews " +
5   "WITH city, pkg, pkg_reviews, CASE WHEN age < 30 THEN '1. Under 30'
   WHEN age >= 30 AND age <= 50 THEN '2. Between 30-50' ELSE '3. Over 50' END
   AS age_group " +
6   "WITH age_group, city.name AS destination, count(pkg) AS
   total_purchases, avg(pkg.avg_rating) AS raw_rating, sum(pkg_reviews) AS
   total_reviews " +
7   "WITH age_group, destination, total_purchases, " +
8   "      (raw_rating * total_reviews + 20.0) / (total_reviews + 5.0) AS
   weighted_rating " +
9   "ORDER BY age_group ASC, total_purchases DESC, weighted_rating DESC "
10  +
    "WITH age_group, collect({city: destination, purchases:
    total_purchases, rating: round(weighted_rating, 1)})[0..3] AS top " +
    "RETURN age_group AS ageRange, " +

```

```

11      "top[0].city + ' (' + toString(top[0].purchases) + ' purchases, score:
12      ' + toString(top[0].rating) + '))' AS firstPlace, " +
13      "coalesce(top[1].city + ' (' + toString(top[1].purchases) + '
14      purchases, score: ' + toString(top[1].rating) + '))', '-') AS secondPlace,
15      " +
16      "coalesce(top[2].city + ' (' + toString(top[2].purchases) + '
17      purchases, score: ' + toString(top[2].rating) + '))', '-') AS thirdPlace "
18      +
19      "ORDER BY ageRange ASC")
20  List<PopularDestinationsDTO> getPopularDestinationsByAgeGroup();

```

Listing 8.9: Cypher Query for Top Destinations by Demographic Cohort

8.2.5 Geospatial Point of Interest (POI) Discovery

This query enables users to expand their existing travel package by discovering and adding new, nearby Points of Interest (POIs). By leveraging Neo4j's native spatial capabilities, it calculates the real-world distance between the hotel associated with the package and other attractions within the city. To ensure optimal performance, the distance condition is structured to trigger Neo4j's spatial indexing, strictly limiting the search to a 5-kilometer radius. Finally, it guarantees that the available options remain relevant by explicitly filtering out any POIs the user has already included in their itinerary.

Each entry in the result contains the following information:

- **poiId**: the unique identifier of the Point of Interest
- **poiName**: the name of the attraction or location
- **category**: the primary classification of the POI (e.g., Museum, Park, Restaurant)
- **longitude**: the exact longitudinal coordinate extracted from the spatial node
- **latitude**: the exact latitudinal coordinate extracted from the spatial node
- **distance**: the calculated distance in meters from the hotel to the POI, cleanly rounded
- **packageId**: the identifier of the reference travel package

Query Execution Description

The Cypher query performs the following steps:

1. **MATCH – Context Initialization**: The query begins by anchoring the search to a specific user (`$userId`) and their purchased travel package (`$packageId`).
2. **MATCH – Graph Traversal to Spatial Nodes**: The algorithm traverses from the travel package to its associated `Hotel`, and subsequently maps out the `CITY` and all `POI` nodes located within that same municipality.
3. **WHERE – Spatial Indexing and Anti-Patterns**: This is the most critical optimization phase. By calculating the distance using `point.distance(p.location, h.location) <= 5000` directly in the `WHERE` clause, Neo4j triggers its spatial index, drastically reducing the search space to a 5km radius. Additionally, it applies two exclusion filters: `NOT (tp)-[:INCLUDES]->(p)` ensures POIs already inside the package are ignored, and `NOT p.id IN $alreadyAddedPoiIds` prevents suggesting items the user has manually queued.

4. **WITH – Distance Projection:** The computed spatial distance is passed forward to the next stages of the pipeline to avoid redundant calculations.
5. **MATCH and WITH – Category Resolution:** The query traverses to the **Category** node associated with the POI. Because a POI might hypothetically belong to multiple categories, the `collect(c.name)[0]` function is utilized as a safety mechanism to extract a single, definitive category name.
6. **RETURN and ORDER BY – Result Formatting:** The final stage prepares the data for the application. It retrieves the POI's basic details and extracts the exact latitude and longitude coordinates so they can be easily displayed on a map. Finally, it sorts the results by distance to show the closest attractions first, limiting the output to the top 15 locations.

```

1 @Query("MATCH (target:User {id: $userId})-[:PURCHASED]->(tp:TravelPackage {id
2   : $packageId}) " +
3   "MATCH (tp)-[:HAS_HOTEL]->(h:Hotel)-[:LOCATED_AT]->(city:CITY)<-[:
4   LOCATED_IN]->(p:POI) " +
5   // 1. SBLOCCO INDICE SPAZIALE: Uso point.distance direttamente nel
6   WHERE
7   "WHERE point.distance(p.location, h.location) <= 5000 " +
8   " AND NOT (tp)-[:INCLUDES]->(p) " +
9   " AND NOT p.id IN $alreadyAddedPoiIds " +
10  // 2. Ora che abbiamo scremato, passiamo la distanza per il RETURN
11  "WITH tp, p, h, point.distance(p.location, h.location) AS distance "
12  +
13  // Troviamo la categoria del POI
14  "MATCH (p)-[:BELONGS_TO]->(c:Category) " +
15  // Raggruppiamo nel caso un POI avesse pi categorie (prendiamo la
16  prima per sicurezza)
17  "WITH p, distance, tp, collect(c.name)[0] AS categoryName " +
18  "RETURN p.id AS poiId, p.name AS poiName, categoryName AS category, "
19  +
20  // Estraiamo le coordinate dal nodo spaziale di Neo4j
21  "      p.location.longitude AS longitude, p.location.latitude AS
22  latitude, " +
23  "      round(distance) AS distance, $packageId AS packageId " +
24  "ORDER BY distance ASC LIMIT 15")
25 List<NearestPoiDTO> getNearestPOIs(
26     @Param("userId") String userId,
27     @Param("packageId") String packageId,
28     @Param("alreadyAddedPoiIds") List<String> alreadyAddedPoiIds);

```

Listing 8.10: Cypher Query for Geospatial Point of Interest Discovery

Chapter 9

Indexes

9.1 MongoDB Indexes

This section explores the indexes that we implemented to improve the performance of read operations on the MongoDB collections. For each candidate index, we evaluated the corresponding application query and compared its execution with and without the index. The comparison was performed using MongoDB Compass, considering the execution time, the number of examined index keys and the number of examined documents.

9.1.1 Reviews Collection

In this collection, in addition to the default index on the `_id` field, a specific index was defined to optimize database performance regarding package-specific queries.

Package_ID

An index on the `package_id` field was added to improve the performance of review retrieval. This operation is frequently executed when users visit a specific travel package's detail page, requiring the application to efficiently load all the associated reviews. At the repository layer, this logic is handled by the Spring Data method:

```
findByPackageId(String packageId)
```

To evaluate the usefulness of this index, we executed a query that retrieves all reviews belonging to a specific package, both with and without the index.

```
{ package_id: ObjectId("7ba504c7c01a4d30557f6fac") }
```

	Without index	With index
Execution Time (ms)	97	13
Keys examined	0	59
Documents examined	37800	59

Table 9.1: Performance comparison for the `package_id` index in the Reviews collection

Without the index, MongoDB was forced to perform a full collection scan (`COLLSCAN`). It examined all 37,800 documents in the `Reviews` collection just to find the 59 matches. By leveraging the `package_id` index (`IXSCAN`), MongoDB only examined the 59 index keys that strictly satisfied the query condition and loaded exclusively the 59 corresponding documents into memory, bypassing the need to scan the entire collection.

The results demonstrate a significant performance improvement: the execution time dropped from 97 ms to 13 ms, and the number of examined documents was drastically reduced to the absolute minimum required. Consequently, the index on the `package_id` field proved to be highly beneficial for the application's scalability and was permanently integrated into the database schema.

9.1.2 Flights Collection

In this collection, a compound index was implemented to optimize the core routing logic of the recommendation engine.

Flight_Search (Compound Index)

To efficiently match users with valid travel packages, the system must first resolve available flight routes. A compound index was created on the `departure.city` and `arrival.city` fields. This specific structure not only accelerates exact route searches, but, thanks to MongoDB's index prefix rule, it simultaneously optimizes preliminary queries filtering solely by the departure city (handled by the `findReachableDestinations` method).

To evaluate its impact, a search for a specific travel route was executed both with and without the index:

```
{ "departure.city": "Rome", "arrival.city": "London" }
```

	Without index	With index
Execution Time (ms)	218	2
Keys examined	0	303
Documents examined	234342	303

Table 9.2: Performance comparison for the `idx_flight_search` compound index

Without the index, MongoDB was forced to execute a massive full collection scan (`COLLSCAN`). The engine had to load and evaluate all 234,342 documents in memory just to isolate the 303 matching flights, taking 218 ms. To understand why this is a severe architectural bottleneck, it is important to consider the overall flow of the recommendation engine: finding available flights is only the preliminary phase of the user's request. The application must first figure out exactly which cities the user can physically reach before it can even begin the core task of filtering, scoring, and suggesting the actual travel packages. If this initial flight search is slow, the entire system is paralyzed, forcing the user to wait before the heavy package aggregation has even started.

By utilizing the compound index (`IXSCAN`), the database efficiently traversed the structured index tree to pinpoint the exact 303 keys, fetching only the strictly necessary documents. The performance optimization is drastic: execution time plummeted from 218 ms to a nearly instantaneous 2 ms, allowing the application to seamlessly pass the list of reachable cities to the subsequent aggregation pipeline with zero delay.

The decision to implement a compound index also involved a careful evaluation of its inherent architectural trade-offs. Compound indexes naturally consume more memory (RAM) because they store multiple fields per node, and they introduce a performance penalty during write operations (inserts and updates). However, in this specific domain context, the benefits overwhelmingly justify the costs. The flights catalog acts as a highly read-intensive dataset that remains largely static, with route updates occurring extremely rarely (e.g., annually when airline schedules change). Consequently, the write penalty is virtually non-existent, and trading memory footprint for a massive reduction in CPU load and query latency represents a well-balanced architectural trade-off.

9.1.3 Travel Packages Collection

In the `TravelPackages` collection, indexing is primarily designed to protect the database from excessive CPU consumption during the dynamic recommendation phase.

City_ID

The core of the recommendation engine relies on a resource-intensive aggregation pipeline (`findTop3BestMatches`) that calculates matching scores based on user interests, season, and travel style. To prevent the database from evaluating irrelevant packages, an initial `$match` stage aggressively filters the dataset by reachable destinations and budget limits.

To optimize this crucial step, a single-field index named `idx_city` was created on the `city` field. We isolated and executed this filtering query with and without the index:

```
{ "city": { "$in": ["London", "Paris", "Madrid", "Berlin"] }, "price": {
                                "$lte": 1500 } }
```

	Without index	With index
Execution Time (ms)	13	16
Keys examined	0	2049
Documents examined	10000	2046

Table 9.3: Performance comparison for the `idx_city` preliminary filter

Without the index, MongoDB executed a `COLLSCAN`, sequentially loading the entire collection of 10,000 documents into memory to evaluate the filtering conditions. By utilizing the index (`IXSCAN`), the engine examined only the 2,049 index keys that strictly satisfied the condition, fetching exactly the 2,046 matching documents.

While the execution times appear comparable in this isolated test (due to the relatively small dataset size of 10,000 documents allowing for an extremely fast in-memory cache scan), the structural necessity of the index is demonstrated by the drastic reduction in documents examined. Database optimization must ensure long-term scalability: as the catalog expands, the cost of a full collection scan would degrade linearly ($\mathcal{O}(N)$), inevitably causing the subsequent scoring logic to become a major bottleneck. The index guarantees that the aggregation pipeline scales efficiently, isolating the working set in logarithmic time ($\mathcal{O}(\log N)$) before initiating any heavy computational load.

9.2 Neo4j Graph Database Indexes

This section explores the indexes that we implemented to improve the performance of read operations on the Neo4j nodes. For each candidate index, we have evaluated the corresponding application query and compared its execution with and without the index. The comparison was performed using Neo4j Desktop with the Cypher keyword `"PROFILE"`, considering the number of nodes and relationships examined and the execution time.

9.2.1 User_Id_Idx

In the `User` nodes, indexing is primarily designed to instantly locate the starting point of the recommendation queries, preventing the database from scanning the entire user base before analyzing the data.

The recommendation filtering engine (such as the `getRecommendationsByInterests` pipeline) systematically begins by searching for a specific user via their unique identifier:

```
MATCH (u:User {id: $userId})
```

To optimize this crucial preliminary phase, a single-field index named `user_id_idx` was created on the `id` property. To evaluate its impact, the recommendation query was profiled both with and without the index.

	Without index	With index
Execution Plan	NodeByLabelScan	NodeIndexSeek
Total DB Hits	134295	104293
Execution Time (ms)	1266	688

Table 9.4: Performance comparison for the `user_id_idx` index

Without the index, the execution plan revealed a `NodeByLabelScan` followed by a `Filter` operation. Neo4j was forced to load into memory and sequentially evaluate all the `User` nodes in the database to find the one matching the requested ID. This inefficiency in locating the initial node brought the total cost of the query to 134,295 database hits in 1,266 milliseconds. By enabling the `user_id_idx` index, the execution plan optimally shifted to a `NodeIndexSeek`. The database completely bypassed the initial sequential scan, locating the starting node directly. This reduced the total operations to 104,293 DB hits, with the execution time nearly halved (688 ms). By eliminating the initial search bottleneck with a logarithmic time complexity operation ($\mathcal{O}(\log N)$), the graph engine can instantly initiate the native exploration of relationships (such as `[ :REVIEWED ]` and `[ :HAS_CATEGORY ]`), guaranteeing the performance of the recommendation query regardless of the number of registered users.

9.2.2 User_Birth_Idx

While the previous index optimizes the search for a specific user, the `getRecommendationsByAge` pipeline requires a different approach. This algorithm dynamically calculates an age bracket based on the target user (e.g., "between 30 and 50 years old") and searches the database for a cohort of other users whose `birth_date` property falls strictly within that precise temporal interval.

To optimize this range query, a single-field index named `user_birth_idx` was created on the `birth_date` property.

To evaluate the impact of this optimization, the query was profiled both with and without the index (keeping the ID index active to strictly isolate the performance related to the age range calculation).

	Without index	With index
Execution Plan	NodeIndexScan + Filter	NodeIndexSeekByRange
Total DB Hits	114843	96604
Execution Time (ms)	1087	787

Table 9.5: Performance comparison for the `user_birth_idx` range index

Without the specific birth date index, the execution plan revealed a `NodeIndexScan` operation followed by a `Filter` stage. Lacking a data structure optimized to search directly within the temporal boundaries, the database was forced to scan all user nodes, subsequently filtering the birth date of each one in memory to verify if it fell within the requested range. This inefficiency generated a high volume of reads, bringing the cost to 114,843 database operations (DB hits) in 1,087 milliseconds.

By enabling the `user_birth_idx` index, the execution plan optimally shifted to a `NodeIndexSeekByRange`. Neo4j leveraged the underlying B-Tree index structure to extract

directly and exclusively the users that fell within the requested temporal period. This reduced the total operations to 96,604 DB hits, lowering the execution time to 787 ms. By resolving the search for the target user cohort in logarithmic time ($\mathcal{O}(\log N)$), the graph engine bypasses a potential linear-complexity bottleneck and can efficiently proceed to traverse the purchase relationships (`[:PURCHASED]`), guaranteeing scalable recommendations.

9.3 Sharding

This section focuses on sharding. Although it was not ultimately implemented in the system, the following paragraphs outline the theoretical strategies and techniques that would have been employed to scale the application in a production scenario.

9.3.1 MongoDB sharding

In MongoDB, we defined different sharding strategies based on the specific access patterns of each collection, strictly differentiating between dynamic user-generated operations and high-frequency read catalog data.

Users Collection

For the Users collection (which incorporates personal information, identity documents, and shopping cart management), a hashed sharding strategy on the `_id` field would have been selected. User interactions are highly targeted: registration, document updates, and checkout rely on direct queries tied to a specific user. Hashing the `_id` guarantees an even distribution of data across the shards, allowing the system to route these high-frequency operations to a single, authoritative shard, thus maintaining extremely low latency.

Questionnaires Collection

For the Questionnaires collection, a hashed sharding strategy on the `user_id` field was theorized. When a user submits the questionnaire, the system immediately triggers the generation of suggested packages. Sharding by `user_id` ensures that the write load generated by questionnaire submissions is uniformly distributed across all cluster nodes, preventing a single server from becoming a bottleneck during traffic spikes (e.g., if many users submit the form simultaneously).

Reviews Collection

For the Reviews collection, the ideal strategy is a hashed sharding on the `package_id` field. Users frequently consult reviews for specific packages, applying filters by rating. By hashing the `package_id`, the database ensures that all reviews related to the same package are stored within the same shard. This prevents expensive broadcast queries: when the application needs to load a package's reviews, it exclusively queries the shard designated for that `package_id`, retrieving the data in a targeted and immediate manner.

Packages and Flights Collections

For the Packages and Flights collections, we determined not to implement sharding. The application makes intensive use of complex aggregation queries on packages to compute the "Popular" and "For You" sections. Applying sharding would force the database to broadcast these read operations across all shards and then merge the results, thereby degrading performance. Furthermore, flights are updated only once a year, presenting a volume of operations that is almost exclusively read-only. For this catalog data, maintenance on a single, robust

Replica Set represents the optimal architectural choice to guarantee maximum read speed, without the computational overhead of a fragmented cluster.

9.4 Deployment and Replica Set Configuration (Local Environment)

9.4.1 MongoDB

This section details the MongoDB database deployment process. To simulate a distributed production environment and ensure high reliability, the system does not rely on a single (standalone) instance; instead, it has been configured using a **Replica Set** consisting of three nodes running on the same machine (localhost).

Infrastructure Preparation

The first step required the physical creation of the directories necessary to host the isolated data files for each of the three instances:

```
mkdir C:\MongoDB\data\r1
mkdir C:\MongoDB\data\r2
mkdir C:\MongoDB\data\r3
```

Subsequently, the three MongoDB servers were started in separate processes. Each node was assigned a dedicated port, a specific data folder, and the directive to join the same cluster named `rs0`:

```
mongod --replSet rs0 --dbpath c:\MongoDB\data\r1 --port 27018 --bind_ip
localhost --oplogSize 200
mongod --replSet rs0 --dbpath c:\MongoDB\data\r2 --port 27019 --bind_ip
localhost --oplogSize 200
mongod --replSet rs0 --dbpath c:\MongoDB\data\r3 --port 27020 --bind_ip
localhost --oplogSize 200
```

Cluster Initialization

To interconnect the isolated instances, a connection was established via the `mongosh` shell to the port of the first node (27018). Within the administration environment, the Replica Set configuration was defined and applied:

```
rsconf = {
  _id: "rs0",
  members: [
    { _id: 0, host: "localhost:27018" },
    { _id: 1, host: "localhost:27019" },
    { _id: 2, host: "localhost:27020" }
  ]
};
rs.initiate(rsconf);
```

Following initialization, the system automatically elects a **Primary** node (on port 27018), which holds the operational data and is exclusively responsible for receiving write operations. The other two nodes assume the role of **Secondary** (on ports 27019 and 27020) and maintain an identical, constantly synchronized copy of the database.

Data Migration

The transition from the standalone development database to the distributed architecture was managed using native export tools. First, a complete dump of the original database was performed:

```
mongodump --port 27017 --db DiscoverEurope --out C:\MongoDB\backup
```

After verifying the operational status of the new, empty Replica Set, the data was restored directly to the Primary node, which then automatically propagated it to the secondary nodes:

```
mongorestore --port 27018 --db DiscoverEurope C:\MongoDB\backup\DiscoverEurope
```

9.4.2 Neo4j

Because the application's data is highly connected and graph queries are computationally intensive, Neo4j is deployed as a single, standalone instance hosted on the local machine (localhost). In fact, unlike document databases, graph databases rely on constantly navigating relationships between nodes. This makes them unsuitable for traditional sharding, because splitting relationships across different machines would cause severe performance drops.

9.4.3 CAP Theorem Considerations

According to the CAP theorem, the system architecture adopts an AP (Availability and Partition Tolerance) model to ensure an uninterrupted user experience during exploration. Since the application sustains a high volume of read-oriented traffic (browsing travel packages, categories, and points of interest), continuous availability is a fundamental requirement.

Availability

The high availability of the system is systematically guaranteed through two main architectural mechanisms, designed to keep the platform highly responsive at all times:

- **Read Workload Distribution:** To prevent bottlenecks, the application leverages the routing capabilities of the MongoDB Replica Set via the `readPreference=secondaryPreferred` directive. The vast majority of read traffic (e.g., browsing `TravelPackage`, filtering by `CATEGORY`, or searching for a `CITY`) is proactively routed to the Secondary nodes. This strategic offloading ensures that catalog exploration never experiences slowdowns, keeping the Primary node completely free to process write operations.
- **Asynchronous Workload Management (Neo4jSyncManager):** The second pillar of availability is the decoupling of writes through the custom `Neo4jSyncManager` service. Updating complex graph relationships (such as `_INCLUDES_` between packages and POIs, or `_HAS_CATEGORY_`) is computationally expensive. Instead of blocking the user while waiting for these operations, the architecture saves the transaction immediately in MongoDB (which acts as the single source of truth) and queues a synchronization task. A scheduled job updates the Neo4j database only if the system CPU load is within safe limits (below 40% Max). By accepting a temporary eventual consistency between the two databases, the system saves precious resources during peak times and never compromises the user's perceived responsiveness.

Partition Tolerance

Partition tolerance ensures that the system survives network failures or individual component crashes, applying different resilience strategies based on the database involved:

- **Replica Set Resilience:** In the event of a Primary node failure or a network partition in MongoDB, the system continues to guarantee data availability by serving reads through the surviving Secondary nodes while waiting for the election of a new leader.
- **Graceful Degradation (Neo4j):** Because Neo4j is deployed as a single instance (on the localhost machine), it lacks native cluster-level partition tolerance. However, the overall architecture is designed to tolerate its failure. Should the Neo4j process crash, the application does not experience a total outage. Complex graph exploration features undergo a temporary degradation, but the core system remains online, continuing to serve critical user and package data by retrieving it from the primary MongoDB database.

9.4.4 Handling Inter-DB Consistency

Integrating two distinct database technologies within the same architecture inherently introduces the challenge of maintaining data consistency across multiple representations. Within our application, the storage roles are strictly partitioned based on the specific strengths of each database:

- **MongoDB** acts as the primary system of record, storing the comprehensive and authoritative models for users, travel packages, questionnaires, flights, and orders.
- **Neo4j** maintains a specialized, graph-oriented projection of this data. It isolates and connects lightweight entities (such as `USER`, `TravelPackage`, `CATEGORY`, `POI`, and `CITY`) to perform complex relationship navigations, such as discovering packages based on categories or mapping included points of interest.

MongoDB and Neo4j Synchronization

The main consistency challenge within the architecture involves MongoDB and Neo4j, since core entities and user interactions (like purchasing or reviewing a package) are distributed across both databases. MongoDB serves as the primary system containing the complete and detailed profiles. In contrast, Neo4j stores a lightweight projection of these entities, keeping the graph storage optimized and tailored exclusively for relationship navigation.

To manage this duality without compromising performance, the system implements an *eventual consistency* approach driven by a custom service named `Neo4jSyncManager`. Write operations that modify shared data—such as registering a new user, purchasing a travel package, or submitting a review—are strictly executed synchronously on MongoDB. This provides immediate feedback to the user (enforced by the `w=1`).

Following a successful write on MongoDB, a background task (encapsulating the specific operation, e.g., `ADD_PURCHASED_RELATION` or `SYNC_NEW_USER`) is delegated to the `Neo4jSyncManager` via a concurrent queue. This manager is responsible for the asynchronous propagation of structural changes to Neo4j. Crucially, the synchronization job executes in total safety by preemptively verifying hardware resources: graph mutations occur only if the system's CPU load is within safe limits (below 40%).

In case of an error, for example, if Neo4j is temporarily unavailable or a network issue occurs, the execution fails and the error is logged. To ensure resilience and prevent data loss between the two databases, the manager intercepts the exception and automatically re-adds the failed task to the queue for a subsequent retry.

This design choice heavily prioritizes high availability and minimizes user-perceived latency. By deferring the graph update, the system inevitably introduces a temporary window of inconsistency. For instance, when a user purchases a package, the order becomes immediately visible in their MongoDB profile, but the corresponding `_PURCHASED_` relationship might momentarily be missing in the Neo4j-driven features. However, this behavior perfectly aligns with the system's AP architecture, and eventual consistency between the document and graph representations is reliably restored as soon as the pending synchronization task successfully executes.

Chapter 10

Use of Artificial Intelligence in Development

It is important to highlight that, during the project's development, Artificial Intelligence tools and Large Language Models were used as a support mechanism rather than a replacement for human decision-making. All generated suggestions were carefully evaluated, adapted, and integrated into the project according to the specific architectural requirements.

The use of AI contributed significantly to improving development efficiency, optimizing database queries, and enhancing code quality, while maintaining full developer control over the system design and implementation choices.

10.1 Overview

10.1.1 Support in Database Population

AI tools helped develop the Python scripts needed to populate and synchronize a part of the system. First, specific populators were written to simulate user questionnaires, cart items, and historical orders. Additionally, AI assistance made it easier to design the migration pipelines to transfer data from MongoDB to Neo4j, helping to create the graph nodes (Users, TravelPackages, Cities, Hotels) and establish the complex web of relationships (PURCHASED, REVIEWED, LOCATED_AT) that power the recommendation engine.

10.1.2 Java and Spring Boot Architecture

Large Language Model tools served as a valuable consultative resource during the architectural design of the background task management system. To handle asynchronous database synchronization between MongoDB and Neo4j, we asked the LLM to identify the optimal Java constructs. This led to the implementation of a thread-safe `ConcurrentLinkedQueue` and the `@Scheduled` annotation. Furthermore, to enforce our custom low-load execution policy, we specifically queried the LLM for a native CPU monitoring built-in function. This highlighted the `OperatingSystemMXBean` interface, which we integrated into a custom handler to guarantee that graph synchronization tasks are processed exclusively when CPU utilization remains below a strict 40% threshold.

Finally, these tools significantly helped in the development workflow by assisting in the interpretation and resolution of complex debugging errors encountered within the IntelliJ IDE, particularly those related to Spring Bean initialization and Circular Dependencies.

10.1.3 Database Synchronization Strategy

LLM tools helped us refine practical solutions for handling distributed data. When deciding how to keep Neo4j updated without slowing down the primary MongoDB transactions, AI support was crucial in validating our Event-Driven architectural choice. It aided in defining the structure of the `TaskToDo` event payloads, ensuring that only the strictly necessary data (such as ObjectIDs, relationship timestamps, and ratings) were queued in memory, preventing potential OutOfMemory (OOM) exceptions during high-traffic checkout phases.

10.1.4 MongoDB and Neo4j Query Optimization

Throughout the database optimization phase, LLM tools were utilized to refine, format, and troubleshoot highly complex queries across both MongoDB and Neo4j, strictly after the underlying logic had been manually drafted.

For MongoDB, this support was particularly valuable in correctly structuring the multi-stage Aggregation Pipeline required for the Veto-Based Scoring Algorithm, assisting with the correct syntax for conditional operators (`$cond`) and array intersections (`$setIntersection`).